

SoftJAX & SoftTorch: Empowering Automatic Differentiation Libraries with Informative Gradients

Anselm Paulus^{*1} A. René Geist^{*1} Vít Musil² Sebastian Hoffmann³ Georg Martius¹⁴

Abstract

Automatic differentiation (AD) frameworks such as JAX and PyTorch have enabled gradient-based optimization for a wide range of scientific fields. Yet, many “hard” primitives in these libraries such as thresholding, Boolean logic, discrete indexing, and sorting operations yield zero or undefined gradients that are not useful for optimization. While numerous “soft” relaxations have been proposed that provide informative gradients, the respective implementations are fragmented across projects, making them difficult to combine and compare. This work introduces SoftJAX and SoftTorch, open-source, feature-complete libraries for *soft differentiable programming*. These libraries provide a variety of soft functions as drop-in replacements for their hard JAX and PyTorch counterparts. This includes (i) elementwise operators such as clip or abs, (ii) utility methods for manipulating Booleans and indices via fuzzy logic, (iii) axiswise operators such as sort or rank – based on optimal transport or permutahedron projections, and (iv) offer full support for straight-through gradient estimation. Overall, SoftJAX and SoftTorch make the toolbox of soft relaxations easily accessible to differentiable programming, as demonstrated through benchmarking and a practical case study. Code is available at github.com/a-paulus/softjax and github.com/a-paulus/softtorch.

1. Introduction

Automatic differentiation (AD) frameworks have en-

^{*} Equal contribution. Libraries implemented by Anselm Paulus.
¹University of Tübingen, Germany ²Masaryk University, Czechia
³Max Planck Institute for Biogeochemistry, Germany ⁴Max Planck Institute for Intelligent Systems, Tübingen, Germany. Correspondence to: Anselm Paulus <anselm.paulus@tuebingen.mpg.de>, A. René Geist <rene.geist@uni-tuebingen.de>.

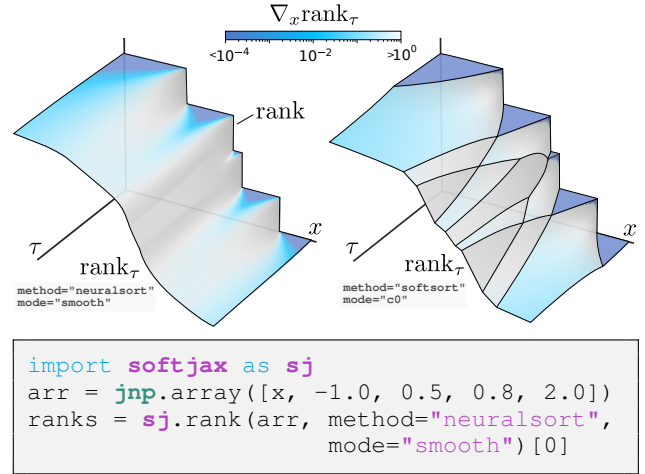


Figure 1. **Top:** SoftJAX and SoftTorch provide differentiable surrogates for discrete operations i. e., rank_τ instead of rank. Numerous soft approximations can be obtained by tuning the softness parameter τ , selecting the softening method (e. g., “neuralsort” or “softsort”), and choosing a smoothness mode (e. g., `smooth` for C^∞ or `c1` for C^1). **Bottom:** The soft surrogates shown above are instantiated using just three lines of code.

abled rapid progress in machine learning. They make gradient computation efficient, user-friendly, and composable. Thereby, they become a ubiquitous tool widely adopted beyond machine learning. This includes (i) *differentiable rendering* (Wang et al., 2019) using Nerfs (Mildenhall et al., 2020) and Gaussian splatting (Kerbl et al., 2023); (ii) *differentiable simulation*, such as MuJoCo XLA (MJX) (Todorov et al., 2012; MuJoCo, 2024; Paulus et al., 2026) and WARP (Howell et al., 2025); (iii) *structured prediction* like ranking (Rolínek et al., 2020a) and matching (Rolínek et al., 2020b); (iv) *combinatorial layers* for CEM (Amos & Yarats, 2020), MPC (Amos et al., 2018), and discrete decisions (Vlastelica et al., 2020; Berthet et al., 2020); (v) *differentiable optimization* (Amos & Kolter, 2017; Agrawal et al., 2019; Blondel et al., 2022); and (vi) *physical simulations*, e. g., for a gravitational wave detector (Ruiz-Gonzalez et al., 2025). However, the programs for these and many more applications contain classical operations using comparisons for branching code, ranking elements, and so forth.

Unfortunately, AD does not necessarily result in informative gradients. Here, *informative* means providing a stable local direction leading to improvements in the

objective. Uninformative are cases with zero gradients (e. g., rounding, comparisons, indexing, sort/max/top-k/median) and arbitrary subgradients (ReLU at zero, sort with duplicates). Many applications call for replacing such hard operations with soft ones that yield informative gradients. A successful example is addressing the “dying ReLU problem” (Lu et al., 2020), where ReLU’s zero gradients hinder gradient-based optimization of deep neural networks, by replacing the ReLU function with a smooth relaxation such as SiLU or Softplus.

While many relaxation techniques have been proposed, the existing toolbox is fragmented. Proposed techniques include analytic smooth surrogates (e. g., SiLU for ReLU (Hendrycks & Gimpel, 2016), sigmoid for steps (Petersen et al., 2021; 2022b), and softmax for argmax (Beker et al., 2025; 2026)), optimization-based approaches (e. g., sorting via regularized optimal transport (Cuturi et al., 2019; Xie et al., 2020; Blondel et al., 2018) or projections onto the permutahedron (Blondel et al., 2020; Sander et al., 2023), proximal methods (Paulus et al., 2024)), stochastic relaxations (e. g., Gumbel-softmax (Maddison et al., 2017; Jang et al., 2017; Paulus et al., 2020), perturbed optimizers (Berthet et al., 2020; Niepert et al., 2021)), gradient “hacks” / estimators (straight-through (Bengio et al., 2013; Sahoo et al., 2023), black-box differentiation (Vlastelica et al., 2020)) and more.

To unify soft relaxations and ease their use, we present SoftJAX and SoftTorch. Our libraries provide soft drop-in replacements for many commonly used hard operations in JAX (Bradbury et al., 2018) and Pytorch (Paszke et al., 2019). In turn, SoftJAX and SoftTorch form easy to use abstraction layers between user code and underlying primitive operations. Our treatment also draws unifying connections, generalizes various techniques, and provides comprehensible “softness” knobs and “mode” families across operations. As an example, see Figure 1 where we plot the soft rank operator for varying softness parameters $\tau > 0$ and softening modes. As for all our relaxations, we recover the original hard operation as $\tau \rightarrow 0^+$. By combining the vast toolbox of soft relaxations, our libraries make *soft differentiable programming* accessible.

We next explain the operators implemented in SoftJAX and SoftTorch and how their hard versions are relaxed to soft counterparts. Section 2 reviews the basics of soft surrogates and straight-through estimation. Section 3 then describes the softening of elementwise operators in SoftJAX and SoftTorch, such as sign or round, via relaxations of the Heaviside step function. Section 4 shows how more involved axiswise operators, such as sort or quantile, can be softened via optimal transport or projections onto the unit simplex or the permutahedron. An overview of all currently implemented functions in SoftJAX is available in Figure 9.

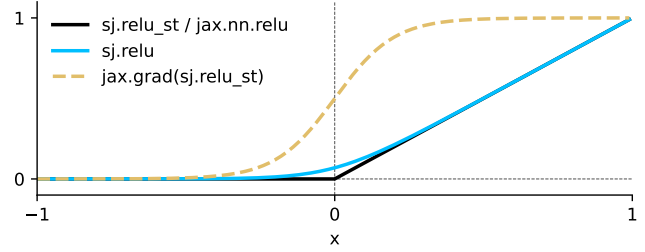


Figure 2. The function `relu_st` resorts to the straight-through trick to use the hard function `jax.nn.relu` in the forward pass while using the soft function `sj.relu` for gradient computation.

Finally, Section 5 presents runtime and memory comparisons to help end users select suitable methods that balance the trade-offs required by their applications.

2. Softening and Straight-Through Estimation

Many functions used in modern ML frameworks are poorly suited for automatic differentiation as they are discontinuous or have large regions with zero derivative. To resolve this, our libraries have one main goal: to provide informative gradients for any program written in the supported frameworks. The underlying mechanism rests on two core concepts: *soft surrogates* and *straight-through estimation*.

Soft surrogate. *Soft surrogates* replace the original function with another function that has more informative gradients for optimization. Specifically, we call the function f_τ with softening parameter $\tau > 0$ a *soft surrogate* of f if (i) f_τ is continuous and differentiable almost everywhere, (ii) f_τ yields informative gradients over the relevant domain, i. e., avoids extended regions of zero derivative, and (iii) f_τ recovers f in the limit $\tau \rightarrow 0^+$. The softening parameter τ controls the trade-off between faithfulness to f and gradient informativeness: larger τ benefits differentiability, while $\tau \rightarrow 0^+$ causes f_τ to approach the original function. See Figure 2 for an example of a soft relu surrogate. Note that softening a function may involve significantly altering the output domain, e. g., a soft surrogate for boolean operations outputs a probability instead of a Boolean value, likewise a soft surrogate for an argmax operation outputs a probability distribution over indices instead of an index.

Straight-through estimation. Unfortunately, blindly replacing a function with a soft surrogate may introduce undesired effects in the forward pass, e. g., resulting in unphysical rollout trajectories of a simulator. Fortunately, these problems can be addressed effectively through *straight-through estimation* (STE).

Historically, STE dates back to early work on single-layer perceptrons (Rosenblatt, 1958), where the derivative of binary activations was replaced with the derivative of the

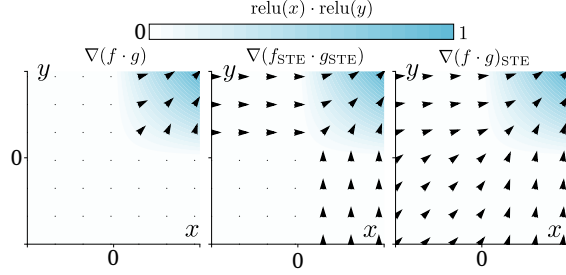


Figure 3. Arrows depict the normalized gradient of the product of two functions $f(x, y) = \text{relu}(x)$ and $g(x, y) = \text{relu}(y)$. Applying straight-through estimation on each function individually causes the gradient $\nabla(f_{\text{STE}} \cdot g_{\text{STE}})$ to be zero if $x < 0$, whereas the gradient $\nabla(f \cdot g)_{\text{STE}}$ is non-zero.

identity map. Following Bengio et al. (2013), STE keeps the original function in the forward pass during automatic differentiation, but uses the gradient of a soft surrogate in the backward pass. In our framework, STE is implemented via the *straight-through trick*, writing

$$f_{\text{STE}}(x) = \text{sg}(f(x)) + f_{\tau}(x) - \text{sg}(f_{\tau}(x)), \quad (1)$$

where sg denotes the stop-gradient operator of an automatic differentiation library. Therefore, we obtain $f_{\text{STE}} = f$ on the forward pass, but $\nabla f_{\text{STE}} = \nabla f_{\tau}$ on the backward pass. In practice, using a soft surrogate for differentiation amounts to wrapping a soft function like `sj.relu` in the `sj.st` function decorator, which implements the straight-through trick, i.e., `y_soft_st = sj.st(sj.relu)`. Figure 2 illustrates straight-through wrapping a soft relu surrogate.

The straight-through pitfall. A subtle issue arises when STE-wrapped functions interact multiplicatively. Automatic differentiation computes the gradient between the product of functions $f_{\text{STE}}(x)$ and $g_{\text{STE}}(y)$ via the product rule as

$$\nabla(f_{\text{STE}} \cdot g_{\text{STE}}) = \nabla f_{\tau} \cdot g + f \cdot \nabla g_{\tau}, \quad (2)$$

where the original functions f and g appear as multiplicative gates in the backward pass. This defeats the purpose of softening, as the gradient can still vanish when scaled by f or g . As illustrated in Figure 3 for the case of $f(x, y) := \text{relu}(x)$ and $g(x, y) = \text{relu}(y)$, $\nabla(f_{\text{STE}} \cdot g_{\text{STE}})$ is zero when $x, y < 0$. To avoid gradient multiplication by zero, STE should be applied to the *composite function* such that

$$\nabla(f \cdot g)_{\text{STE}} = \nabla(f_{\tau} \cdot g_{\tau}) = \nabla f_{\tau} \cdot g_{\tau} + f_{\tau} \cdot \nabla g_{\tau}. \quad (3)$$

In code, this reads: (notice the decorator)

```
@sj.st
def relu_prod(x, y, **kwargs):
    relu_x = sj.relu(x, **kwargs)
    relu_y = sj.relu(y, **kwargs)
    return relu_x * relu_y
```

To our knowledge, this “STE pitfall” and its remedy have not been explicitly discussed before.

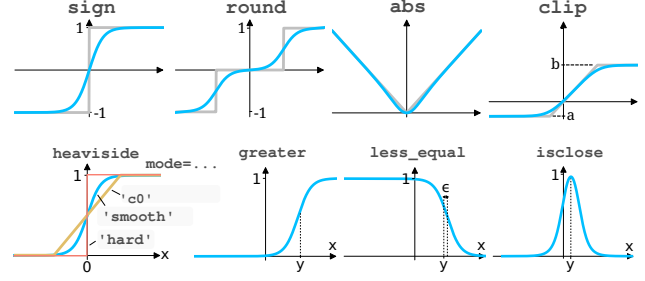


Figure 4. **Top:** The heaviside function surrogates H_{τ} are used to derive the sign, round, abs, and clip functions. **Bottom:** Comparison operations compare values $x, y \in \mathbb{R}$ by interpreting H_{τ} as a CDF defining the probability that $x > 0$.

3. Elementwise Operators

Conceptually, all our elementwise soft operators, some of which are illustrated in Figure 4, are rooted in the relaxation of the *Heaviside step function*, defined as

$$H(x) := \begin{cases} 0 & \text{if } x < 0, \\ 0.5 & \text{if } x = 0, \\ 1 & \text{else.} \end{cases} \quad (4)$$

Its derivative is zero everywhere except at the origin, where it is not defined. Following many previous works (Funahashi, 1989; Han & Moraga, 1995; Rojas, 1996; Mhaskar, 1996; Duch & Jankowski, 1999; Huang et al., 2006b; Yuen et al., 2021; Li et al., 2021), we soften the Heaviside function with a sigmoidal function that traces a characteristic S-shape. Some examples include the standard exponential-based sigmoid (referred to as *smooth mode*)

$$H_{\tau}(x) := \sigma\left(\frac{x}{\tau}\right) = \frac{1}{1 + \exp(-x/\tau)}, \quad (5)$$

or the piece-wise polynomial-based sigmoids¹

$$H_{\tau}(x) := \begin{cases} 0 & \text{if } x < -\tau, \\ g\left(\frac{x}{\tau}\right) & \text{if } -\tau \leq x \leq \tau, \\ 1 & \text{else,} \end{cases} \quad (6)$$

with $g_{c0}(s) := \frac{1}{2} + \frac{s}{2}$, $g_{c1}(s) := \frac{1}{2} + \frac{3s}{4} - \frac{s^3}{4}$, and $g_{c2}(s) := \frac{1}{2} + \frac{15s}{16} - \frac{5s^3}{8} + \frac{3s^5}{16}$. The piecewise modes transition on $[-\tau, \tau]$; in the library, their input is rescaled by $1/5$ so that all modes share an effective transition width of approximately 10τ , matching the smooth mode ($\sigma(\pm 5) \approx 0.007/0.993$). These piece-wise Heaviside relaxations are either continuous (linear, also referred to as *c0 mode*), differentiable (*c1*), or twice differentiable (*c2*).

3.1. Sign, abs, and round

A soft surrogate for the sign function is obtained by the Heaviside function and subtracting a constant. Moreover, a

¹We overload the notation here.

soft surrogate for the abs function is obtained by gating the soft sign function with x , reading

$$\text{sign}_\tau(x) := 2 \cdot H_\tau(x) - 1, \quad (7)$$

$$\text{abs}_\tau(x) := \text{sign}_\tau(x) \cdot x. \quad (8)$$

Following [Semenov \(2025\)](#), the round function is implemented as the difference between shifted sigmoidals

$$\text{round}_\tau(x) := \sum_{k=\lfloor x \rfloor - K}^{\lfloor x \rfloor + K} k [H_\tau(x - k + \frac{1}{2}) - H_\tau(x - k - \frac{1}{2})],$$

where K controls the number of neighbouring sigmoidals that are taken into account. Larger temperatures τ require a higher K , as each sigmoidal is softened over a larger region, thereby increasing its region of influence.

3.2. ReLU and clip

Due to its widespread use in ML, the rectified linear unit (ReLU) deserves extra attention. The function $\text{relu}(x) := \max\{0, x\}$ has zero gradient for $x < 0$, and at $x = 0$ it is only sub-differentiable. To get a soft surrogate, we can transform the sigmoidal via Integration

$$\text{relu}_\tau(x) := \int_0^x H_\tau(t) dt, \quad (9)$$

or via a gating mechanism

$$\text{relu}_\tau(x) := x \cdot H_\tau(x). \quad (10)$$

Therefore, for each of the Heaviside relaxations, we define two ReLU relaxations, some of which are well known. For instance, integrating the sigmoid yields the Softplus $\tau \log(1 + e^{x/\tau})$. On the other hand, gating the sigmoid recovers the SiLU activation ([Hendrycks & Gimpel, 2016](#)) $x \cdot \sigma(x/\tau)$. From the soft ReLU, we can now derive a soft surrogate for the clip function as

$$\text{clip}_\tau(x, a, b) := a + \text{relu}_\tau(x - a) - \text{relu}_\tau(x - b). \quad (11)$$

3.3. Comparison operators and differentiable logic

Next, we turn to soft surrogates of comparison operators, such as greater or equal. The hard operators output a Boolean variable $b \in \{\text{True}, \text{False}\}$, whereas our soft surrogates output a value in the interval $[0, 1]$. We directly get the soft comparison operators via the soft Heaviside function

$$\begin{aligned} \text{greater}_\tau(x, y) &= H_\tau(x - y - \epsilon), \\ \text{gtr_equal}_\tau(x, y) &= H_\tau(x - y + \epsilon), \\ \text{less}_\tau(x, y) &= H_\tau(y - x - \epsilon), \\ \text{less_equal}_\tau(x, y) &= H_\tau(y - x + \epsilon), \\ \text{equal}_\tau(x, y) &= 2 \cdot \text{less_equal}_\tau(\text{abs}_\tau(x - y), 0), \\ \text{isclose}_\tau(x, y) &= 2 \cdot \text{less_equal}_\tau(\text{abs}_\tau(x - y), \text{tol}). \end{aligned} \quad (12)$$

Here, ϵ denotes the machine precision to ensure that the surrogates converge to the correct hard function as $\tau \rightarrow 0^+$. In the definition of isclose_τ , we use a standard tolerance $\text{tol} = \text{atol} + \text{rtol} \cdot \text{abs}_\tau(y)$ with $\text{atol} > 0$ and $\text{rtol} > 0$ denoting the relative and absolute tolerance.

The output of our soft surrogates can be interpreted as the probability of the hard comparison being True. For the sake of simplicity, we refer to such a probability as a soft Boolean or, in short, a ‘‘SoftBool’’. This perspective has been studied extensively in the field of fuzzy logic ([Belohlavek et al., 2017](#)). Given multiple SoftBools $p_1, \dots, p_n$, differentiable logic operators reduce to the manipulation of probabilities. By default, our framework implements the logical all operator as

$$\text{all}(p_1, \dots, p_n) := \prod_{j=1}^n p_j, \quad (13)$$

denoting the joint probability that independent Boolean events $p_1, \dots, p_n$ equate to True. Alternatively, we allow using the geometric mean providing advantageous numerical scaling for gradient computation. Similarly, the not operator yields the probability that a Boolean event is False

$$\text{not}(p) := \neg p = 1 - p. \quad (14)$$

Other fuzzy logical operators are derived using the all and not operators

$$\begin{aligned} \text{any}(p_1, \dots, p_n) &:= \neg \text{all}(\neg p_1, \dots, \neg p_n), \\ \text{and}(p, q) &:= \text{all}(p, q), \\ \text{or}(p, q) &:= \text{any}(p, q), \\ \text{xor}(p, q) &:= \text{or}(\text{and}(p, \neg q), \text{and}(\neg p, q)). \end{aligned} \quad (15)$$

Similar to how normal Boolean variables can be used to select elements among two arrays, we can use a SoftBool variable p_i to do a soft selection via the expectation

$$z_i = p_i \cdot x_i + (1 - p_i) \cdot y_i. \quad (16)$$

In code, this is abstracted from the user via

```
soft_cond = sj.greater(x, y) # [0.05 0.73 0.98]
z = sj.where(soft_cond, x, y) # [0.39 0.27 0.69]
```

4. Axiswise Operators

After discussing elementwise operators in the previous section, this section discusses and derives soft surrogates for axis-wise operators such as argmax, sort or argtopk.

The simplest example of an axiswise operation is the (arg)max operator. The standard argmax operator returns an index $j \in \{0, \dots, n - 1\}$, and the max operator simply selects the corresponding element x_j . Another way to write

this is as an inner product $x_j = \mathbf{e}_j \cdot \mathbf{x}$, where $\mathbf{e}_j$ is one at index j and zero everywhere else. We can now relax the notion of an index as an indicator vector to a “SoftIndex” on the unit simplex

$$\Delta_n := \{\mathbf{p} \in [0, 1]^n \mid \sum_{0 \leq j < n} p_j = 1\}. \quad (17)$$

In the case of the argmax operator, a simple relaxation that produces a SoftIndex is the softmax function (more accurately referred to as *softargmax*). The previous dot-product index-selection can now be interpreted as an expectation

$$\mathbf{p} \cdot \mathbf{x} = \sum_{0 \leq j < n} p_j x_j = \mathbb{E}_{j \sim \mathbf{p}} [x_j]. \quad (18)$$

In code, our drop-in replacements read as:

```
x = jnp.array([0.1, 0.4, 0.8])
idx = jnp.argmax(x) # 2
y = jax.lax.dynamic_index_in_dim(x, idx) # [0.8]

soft_idx = sj.argmax(x) # [0.004 0.042 0.953]
y = sj.dynamic_index_in_dim(x, soft_idx) # [0.78]
```

We can further generalize this technique of using SoftIndices to sorting and ranking. To see this, we first note that sorting an array $\mathbf{x} \in \mathbb{R}^n$ can be viewed as multiplying it with the sorting permutation matrix

$$P^* \in \Sigma = \{P \in \{0, 1\}^{n \times n} \mid P\mathbf{1}_n = \mathbf{1}_n, P^\top \mathbf{1}_n = \mathbf{1}_n\} \quad (19)$$

such that

$$\text{sort}(\mathbf{x}) = P^* \mathbf{x}. \quad (20)$$

Throughout, sort is in ascending order. The rows of P^* are indicator vectors, and the matrix multiplication can be interpreted as doing index selection in parallel. In this sense, the permutation matrix P^* can be interpreted as the generalized argsort operator, i. e.,

$$\text{arg sort}(\mathbf{x}) = P^*. \quad (21)$$

Similarly, the same matrix can be used for ranking, where rank 1 is assigned to the largest element,

$$\text{rank}(\mathbf{x}) = (P^*)^\top [n, \dots, 1]^\top. \quad (22)$$

We can now relax the notion of a permutation matrix into a bistochastic matrix, the set of which is also called the Birkhoff polytope

$$\mathbb{B}_n = \{P_\tau \in \mathbb{R}_+^{n \times n} \mid P_\tau \mathbf{1}_n = \mathbf{1}_n, P_\tau^\top \mathbf{1}_n = \mathbf{1}_n\}. \quad (23)$$

Note that in practice, for sorting or ranking, we only need P_τ to be a row- or column-stochastic matrix, respectively. The following two sections discuss how we can compute

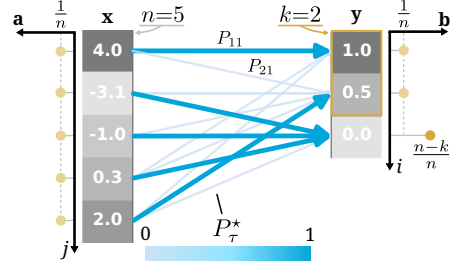


Figure 5. OT-based arg topk computes a (scaled) optimal transport plan P_τ^* whose entries P_{ij}^* contain the probability mass transported from the j -th entry of $\mathbf{x}$ to the i -th entry of the anchor $\mathbf{y}$.

soft permutation matrices P_τ from $\mathbf{x}$, either resorting to regularized optimal transport (OT) or unit simplex projections. Finally, we will discuss an approach that directly relaxes the sort operator via projection onto the permutahedron without relaxing permutation matrices. In this version of the library, we deliberately choose to focus on deterministic approaches, and do not consider stochastic ones such as (Berthet et al., 2020; Jang et al., 2017; Maddison et al., 2017; Paulus et al., 2020). Table 2 provides an overview on the various available methods and regularizations.

4.1. Optimal Transport-based

The first family of algorithms is based on optimal transport. In general, optimal transport describes optimally moving probability mass from one distribution to another distribution while paying minimal cost. As described by Cuturi et al. (2019), regularized OT can be used to define soft (arg)sort operators. The general idea is to mimic a sorting-like behavior by defining two uniform distribution, one on the to-be-sorted input vector $\mathbf{x} \in \mathbb{R}^n$ and one on a set of increasing “anchor” points $\mathbf{y} \in \mathbb{R}^n$. The optimal transport cost matrix is then defined as the squared distance between input and anchor points, which means that probability mass from large values of $\mathbf{x}$ will be moved to large anchor points. As the anchor points are sorted, this creates a sorting behavior, which is illustrated in Figure 5.

OT problem. The regularized optimal transport problem between two discrete probability measures $\sum_{i=1}^n a_i \delta_{x_i}$, with source vector $\mathbf{x} = [x_1, x_2, \dots, x_n]^\top$ and probability weight vector $\mathbf{a} = [a_1, a_2, \dots, a_n]^\top$, and $\sum_{j=1}^m b_j \delta_{y_j}$, with target vector $\mathbf{y} = [y_1, y_2, \dots, y_m]^\top$ and probability weight vector $\mathbf{b} = [b_1, b_2, \dots, b_m]^\top$, is defined as

$$\Gamma_\tau^* := \arg \min_{\Gamma \in U(\mathbf{a}, \mathbf{b})} \langle \Gamma, C_{\mathbf{xy}} \rangle + \tau R(\Gamma). \quad (24)$$

Here, the cost matrix $(C_{\mathbf{xy}})_{ij} = (x_i - y_j)^2$ is computed as the squared differences and U denotes the transport polytope of matrices summing to the marginals across rows and columns, i. e.,

$$U(\mathbf{a}, \mathbf{b}) := \{\Gamma \in \mathbb{R}_+^{n \times m} \mid \Gamma \mathbf{1}_m = \mathbf{a}, \Gamma^\top \mathbf{1}_n = \mathbf{b}\}. \quad (25)$$

Note that we omit the dependency of the optimal transport plan Γ_τ^* on $(\mathbf{a}, \mathbf{x}, \mathbf{b}, \mathbf{y})$ for brevity.

We consider the case of an entropic regularizer $R(\Gamma) = \sum_{i,j} \Gamma_{ij} (\log \Gamma_{ij} - 1)$ (Cuturi, 2013), Euclidean regularizer $R(\Gamma) = \frac{1}{2} \sum_{i,j} \Gamma_{ij}^2$ (Blondel et al., 2018) and p-norm regularizer $R(\Gamma) = \frac{1}{p} \sum_{i,j} |\Gamma_{ij}|^p$ (Paty & Cuturi, 2020). It is well known that the Euclidean regularizer has a sparsity-inducing effect on the gradients; see, e. g., SparseMax (Martins & Astudillo, 2016). As discussed by Sander et al. (2023) in the context of projections onto the permutahedron, the p-norm regularizer shares this property, but additionally allows for everywhere differentiable relaxations (Theorem 1). Interestingly, for univariate inputs, the closed-form solutions of the entropic and Euclidean OT problem reduce to the corresponding sigmoidal functions used in our soft Heaviside relaxation, as shown in Section D.3.

OT based operators. We set the source marginal to a uniform distribution $\mathbf{a} = \mathbf{1}_n/n$, and specifically for sorting and ranking, we follow Cuturi et al. (2019) by setting uniform $\mathbf{b} = \mathbf{1}_n/n$ and grid-like $\mathbf{y} = \frac{1}{n-1}[0, \dots, n-1]^\top$.

We define the scaled transposed transportation plan $P_\tau^* := n(\Gamma_\tau^*)^\top$. Since $\Gamma_\tau^* \mathbf{1} = \mathbf{a}$ and $(\Gamma_\tau^*)^\top \mathbf{1} = \mathbf{b}$, each row and column of P_τ^* sums to 1. This allows us to interpret P_τ^* as a soft permutation matrix.

Moreover, we first standardize and then squash $\mathbf{x}$ with a sigmoid to the range $[0, 1]$. Although this is not strictly necessary, the use of $\mathbf{x}$ and $\mathbf{y}$ in the range $[0, 1]$ improves numerical stability and makes the softening parameter independent of the scale of $\mathbf{x}$. Then we have²

$$\arg \text{sort}_\tau(\mathbf{x}) = P_\tau^* \in \mathbb{R}^{n \times n}, \quad (26)$$

$$\text{sort}_\tau(\mathbf{x}) = P_\tau^* \mathbf{x} \in \mathbb{R}^n, \quad (27)$$

$$\text{rank}_\tau(\mathbf{x}) = (P_\tau^*)^\top [n, \dots, 1]^\top \in \mathbb{R}^n. \quad (28)$$

To abstract the soft indices away from the user, we offer several replacements of utility functions for index selection that mirror the behavior of standard hard JAX and PyTorch index selection, e. g.,

```
x = jnp.array([0.3, 1.0, -0.5])
ind = jnp.argsort(x) # [2 0 1]
val = jnp.take_along_axis(x, ind) # [-0.5 0.3 1.0]

soft_ind = sj.argsort(x)
# [[0.07 0. 0.93], [...], [...]]
val = sj.take_along_axis(x, soft_ind)
# [-0.444 0.31 0.936]
```

In principle sorting is enough to define many other operators like max, median and top-k, via appropriate selection and

²Note, that the multiplication of the soft permutation matrix with $\mathbf{x}$ has close ties to the gating-style definition of the soft relu function in Equation (10), see Section D.5 for details.

combination of the soft-sorted values. However, the dimensionality of the cost matrix for an n -dimensional vector $\mathbf{x}$ is $n \times n$, which can lead to large memory requirements.

Fortunately, it is possible to strongly reduce the dimensionality, by transporting multiple elements of $\mathbf{x}$ which we are not interested in (e. g., the bottom $n - k$ elements in top-k) to a shared dummy anchor point, thereby reducing the number of anchors. For top-k, we follow Xie et al. (2020) by setting $\mathbf{b} = [\frac{1}{n}, \dots, \frac{1}{n}, \frac{n-k}{n}]^\top \in \mathbb{R}^{1 \times (k+1)}$, $\mathbf{y} = \frac{1}{k}[k, \dots, 1, 0]^\top \in \mathbb{R}^{1 \times (k+1)}$, then

$$\arg \text{topk}_\tau(\mathbf{x}, k) = (P_\tau^*)_{1:k} \in \mathbb{R}^{k \times n}, \quad (29)$$

$$\text{topk}_\tau(\mathbf{x}, k) = (P_\tau^*)_{1:k} \mathbf{x} \in \mathbb{R}^k. \quad (30)$$

Note that we can also trivially define an OT-based max as $\text{topk}_\tau(\mathbf{x}, k = 1)$; and min by flipping signs. For q-quantiles we slightly vary the approach of Cuturi et al. (2019) by setting $k = \lfloor q(n - 1) \rfloor$, $\mathbf{b} = [\frac{k}{n}, \frac{1}{n}, \frac{1}{n}, \frac{n-k-2}{n}]^\top$, $\mathbf{y} = \frac{1}{3}[0, 1, 2, 3]^\top$, then we can compute the lower $\downarrow$ and upper $\uparrow$ q-quantiles via the second and third entry of P_τ^* , i. e.,

$$\text{quantile}_\tau^\downarrow(\mathbf{x}, q) = (P_\tau^*)_{2\mathbf{x}} \in \mathbb{R}, \quad (31)$$

$$\text{quantile}_\tau^\uparrow(\mathbf{x}, q) = (P_\tau^*)_{3\mathbf{x}} \in \mathbb{R}. \quad (32)$$

Various quantile definitions can be recovered by appropriately combining the upper and lower quantiles (e. g., interpolating or using midpoint). We also trivially get the median by evaluating the quantile function with $q = 0.5$.

4.2. Approximate OT: unit simplex projection-based

The relaxations based on OT have many desirable properties such as smoothness in the case of an entropic regularizer. However, solving the OT problem can be memory- and compute-intensive in practice. We now describe different soft surrogates based on unit-simplex projections, which have a closed-form solution. The downside is that these relaxations typically still allow for some non-differentiabilities, e. g., when ties occur. However, in practice they often perform well and are therefore selected as the default method of choice.

Unit simplex projection. The regularized linear program over the simplex, see Equation (17), is defined as

$$\Pi_\tau(\mathbf{x}) := \arg \max_{\mathbf{p} \in \Delta_n} \langle \mathbf{x}, \mathbf{p} \rangle - \tau R(\mathbf{p}). \quad (33)$$

We consider Euclidean regularization $R(\mathbf{p}) = \frac{1}{2} \|\mathbf{p}\|_2^2$, which leads to a Euclidean projection onto the unit simplex and entropic regularization $R(\mathbf{p}) = \sum_j p_j \log p_j$, leading to a closed-form solution via the softmax $\exp(\mathbf{x}/\tau) / \sum_j \exp(x_j/\tau)$. Finally, we consider p-norm regularization $R(\mathbf{p}) = \frac{1}{p} \sum_j |p_j|^p$, which leads to a Bregman projection (Bregman, 1967) onto the unit simplex.

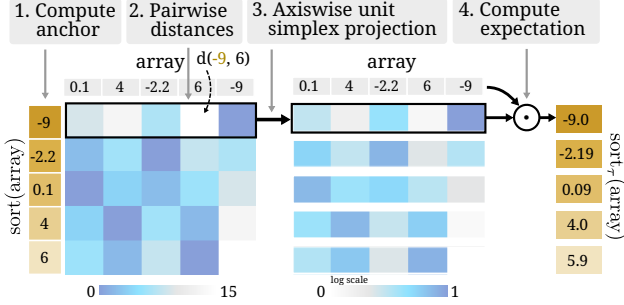


Figure 6. Illustration of the Softsort algorithm.

SoftSort. Entropy-regularized OT can be solved efficiently via the Sinkhorn algorithm (Cuturi, 2013). In the case of OT for sorting in Equation (26), where both marginals $\mathbf{a}$ and $\mathbf{b}$ are uniform, the Sinkhorn algorithm boils down to alternating row and column softmax-normalization of the cost matrix. SoftSort (Prillo & Eisenschlos, 2020) can be seen as approximating this algorithm by reducing it to a single row-wise softmax normalization via a clever initialization of the anchors. This corresponds to the entropic regularizer in Equation (33), which we generalize to other regularizers, leading to new variants of SoftSort.

To define the anchors, instead of using a uniform grid over the interval $[0, 1]$ as in Equation (26), the anchors are constructed by first applying the hard operator to x , e.g., $\mathbf{y} = \text{sort}(\mathbf{x})$. The cost matrix is the absolute difference between elements of $\mathbf{x}$ and $\mathbf{y}$, which will intuitively assign a low transportation cost between elements of $\text{sort}(\mathbf{x})$ that are close to $\mathbf{x}$. For the sort operator, this leads to

$$\arg \text{sort}_\tau(\mathbf{x}) := \Pi_\tau \left(-|\text{sort}(\mathbf{x})\mathbf{1}_n^\top - \mathbf{1}_n\mathbf{x}^\top| \right). \quad (34)$$

The projection is applied row-wise. The computation is illustrated in Figure 6. We showcase the output and partial derivatives of the soft sort operation for two different regularizers over a range of τ in Figure 7.

Note that the computation is independent over rows, we can therefore independently get soft operators for each of the sorted elements, which trivially gives us a soft (arg)topk/quantile/max operator, by replacing the sort function in Equation (34) with the corresponding operator. Interestingly, the argmax operator reduces to $P_\tau(\mathbf{x})$, which for entropic regularization is simply softmax, thereby recovering the example used in the beginning of the chapter. For Euclidean regularization, it recovers the SparseMax (Martins & Astudillo, 2016) operator.

Finally, we can also define a novel SoftSort-style rank operator. For this we observe that Π_τ is applied independently to each row of the cost matrix, so the resulting matrix is only row-stochastic and the corresponding argsort operator cannot be transposed to define a soft ranking as we did in the OT case. Instead, we transpose the cost matrix before applying the unit-simplex projection, which gives distributions

over rank indices as

$$\arg \text{rank}_\tau(\mathbf{x}) := \Pi_\tau \left(-|\mathbf{1}_n \text{sort}(\mathbf{x})^\top - \mathbf{x}\mathbf{1}_n^\top| \right). \quad (35)$$

Note, that all SoftSort-based surrogates are not fully smooth, as the computation involves the hard sort operator (non-smooth at ties) and the absolute value function (non-smooth at zero). For this reason, the NeuralSort-based surrogates (which are fully smooth when combined with our soft abs) are selected as the default method for sort, rank, quantile, and median. However, for argmax, argmin, max, and min, the SoftSort-based method is the default, since it requires only a single simplex projection, giving $O(n)$ complexity compared to NeuralSort’s $O(n^2)$, and in `smooth` mode it reduces to the standard softmax, the canonical soft argmax.

NeuralSort. NeuralSort (Grover et al., 2019) also utilizes unit-simplex projections but originates in a soft relaxation of the median operator. Defining the matrix of soft absolute differences $A_\mathbf{x}^\tau \in \mathbb{R}^{n \times n}$ using our soft abs from Equation (8) as

$$(A_\mathbf{x}^\tau)_{ij} := \text{abs}_\tau(x_i - x_j), \quad (36)$$

we have that $A_\mathbf{x}^\tau \mathbf{1}_n$ is the vector of sums of soft absolute differences between each element and all other elements.³ It is well-known that the median minimizes this sum for $\tau \rightarrow 0^+$, therefore we have

$$\arg \text{median}(x) := \arg \min_{\mathbf{p} \in \Delta_n} \langle A_\mathbf{x}^0 \mathbf{1}_n, \mathbf{p} \rangle. \quad (37)$$

By adding a regularizer similar to SoftSort Equation (33), we get a soft surrogate

$$\arg \text{median}_\tau(x) := \arg \min_{\mathbf{p} \in \Delta_n} \langle A_\mathbf{x}^\tau \mathbf{1}_n, \mathbf{p} \rangle + \tau R(\mathbf{p}) \quad (38)$$

$$= \Pi_\tau(-A_\mathbf{x}^\tau \mathbf{1}_n). \quad (39)$$

NeuralSort generalizes this idea to the sorting operator, by setting

$$\arg \text{sort}_\tau(\mathbf{x})_i := \Pi_\tau \left((2i - n - 1)\mathbf{x} - A_\mathbf{x}^\tau \mathbf{1}_n \right). \quad (40)$$

For $\tau \rightarrow 0^+$ this recovers the true argsort operator. Intuitively, the vector that is projected is a linear combination of the original vector $\mathbf{x}$ and the vector of sums of soft absolute differences, which, when projected onto the unit simplex individually, give an argmax and an argmedian relaxation, respectively.

This also directly provides relaxations for the argmax, argmin, argquantile[†] and argquantile[‡] operators by computing only the corresponding elements of the soft sorted

³The original NeuralSort paper (Grover et al., 2019) uses the hard $|x_i - x_j|$, which introduces gradient discontinuities at ties. By using our soft abs, we obtain fully smooth NeuralSort surrogates.

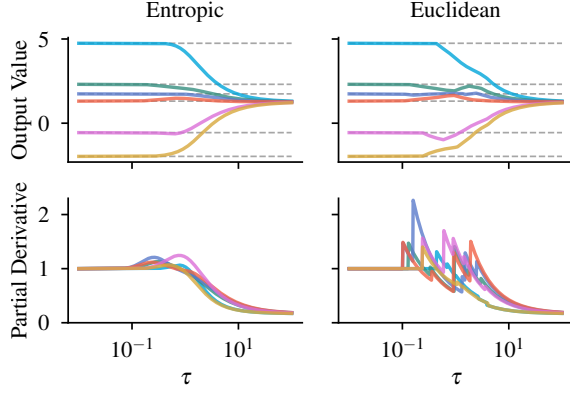


Figure 7. **Top:** Output of the sort operator using “softsort” for an array of $n = 6$ values as a function of softness τ . Elements converge to the mean value as $\tau \rightarrow \infty$. **Bottom:** Partial derivatives $\partial y_j / \partial x_i$. Here, j denotes the sorted position of input element x_i . As $\tau \rightarrow \infty$, the derivatives converge to $1/n$.

array, i.e., with $c^\uparrow = n - 1 - 2\lceil q(n-1) \rceil$ and $c^\downarrow = n - 1 - 2\lfloor q(n-1) \rfloor$,

$$\arg \max_\tau(x) := \Pi_\tau((n-1)\mathbf{x} - A_\mathbf{x}^\tau \mathbf{1}_n) \quad (41)$$

$$\arg \min_\tau(x) := \Pi_\tau(-(n-1)\mathbf{x} - A_\mathbf{x}^\tau \mathbf{1}_n) \quad (42)$$

$$\arg \text{quantile}_\tau^\uparrow(x, q) := \Pi_\tau(-c^\uparrow \mathbf{x} - A_\mathbf{x}^\tau \mathbf{1}_n) \quad (43)$$

$$\arg \text{quantile}_\tau^\downarrow(x, q) := \Pi_\tau(-c^\downarrow \mathbf{x} - A_\mathbf{x}^\tau \mathbf{1}_n) \quad (44)$$

Note that the original paper (Grover et al., 2019) only treats the `smooth` (entropic) projection; we extend it to `c0`, `c1`, and `c2` regularizers. As with SoftSort, the NeuralSort argsort matrix is only row-stochastic. To obtain soft ranks, we normalize each column to sum to one and compute $\text{rank}_\tau(\mathbf{x}) = \tilde{P}^\top[n, \dots, 1]^\top$, mirroring Equation (26).

4.3. Permutahedron projection-based

For sorting and ranking, both OT-based and unit-simplex-projection-based approaches involve computations with the $n \times n$ cost matrix. In SoftJAX, we avoid materializing this matrix by processing rows in chunks via `lax.scan`, reducing memory from $O(n^2)$ to $O(n)$ while also yielding $O(1)$ XLA compilation time. Nevertheless, for large n , the $O(n^2)$ time complexity remains a bottleneck. Blondel et al. (2020); Sander et al. (2023) resolve this issue by directly softening the value-space operators, e.g., `sort` instead of `argsort`. This is achieved by interpreting these operators as projections onto the permutahedron, the convex hull of permutations of $\mathbf{z}$, i.e.,

$$\mathcal{P}(\mathbf{z}) := \text{conv}(\{\mathbf{z}_\sigma \mid \sigma \in \Sigma\}). \quad (45)$$

Here, Σ is the set of permutation matrices, see Equation (19). The Euclidean projection of a vector $\mathbf{y} \in \mathbb{R}^n$ onto the

permutahedron of another vector $\mathbf{z} \in \mathbb{R}^n$ is given by

$$\text{Proj}_\tau(\mathbf{y}, \mathbf{z}) := \arg \max_{\mathbf{p} \in \mathcal{P}(\mathbf{z})} \langle \mathbf{y}, \mathbf{p} \rangle - \frac{\tau}{2} \sum_j p_j^2 \quad (46)$$

$$= \arg \min_{\mathbf{p} \in \mathcal{P}(\mathbf{z})} \frac{1}{2} \left\| \mathbf{p} - \frac{\mathbf{y}}{\tau} \right\|_2^2. \quad (47)$$

Blondel et al. (2020) also propose a variant that uses an entropic regularizer by optimizing over exponentiated inputs and taking the log, i.e.,

$$\text{Proj}_\tau(\mathbf{y}, \mathbf{z}) := \log \left(\arg \max_{\mathbf{p} \in \mathcal{P}(e^\mathbf{z})} \langle \mathbf{y}, \mathbf{p} \rangle - \tau \langle \mathbf{p}, \log \mathbf{p} - \mathbf{1} \rangle \right).$$

FastSoftSort. Intuitively, FastSoftSort (Blondel et al., 2020) now projects the sorted ranks onto the permutahedron of the input $\mathbf{x}$, which will select the permutation of $\mathbf{x}$ that is closest to being sorted. In the same way, projecting the (negative) $\mathbf{x}$ onto the permutahedron of the possible ranks selects the ranks that follow the ordering of $\mathbf{x}$. This gives the surrogate operators

$$\text{sort}_\tau(\mathbf{x}) = \text{Proj}_\tau([1, \dots, n], \mathbf{x}), \quad (48)$$

$$\text{rank}_\tau(\mathbf{x}) = \text{Proj}_\tau(-\mathbf{x}, [1, \dots, n]). \quad (49)$$

Other operators like top-k, quantile, median or max can be computed by first running soft sorting, and then selecting the appropriate values.

Crucially, Blondel et al. (2020) derive an algorithm based on isotonic optimization which solves the projection onto the permutahedron in $O(n \log n)$. This is much faster than the runtime of Sinkhorn which is $O(Tn^2)$, where T is the number of Sinkhorn iterations. However, note that this approach does not allow for an `argsort` or `argrank` operator.

SmoothSort. In OT and unit simplex projection-based approaches, entropic regularization typically leads to dense Jacobians, whereas Euclidean regularization promotes sparsity, see e.g., (Blondel et al., 2018). However, in FastSoftSort this is not the case: While Euclidean regularization behaves as usual by promoting sparsity, the entropic regularization behaves very similarly to the Euclidean case by promoting sparsity, as also discussed by Blondel et al. (2020).

Therefore, we propose a new variant of this surrogate family by adding entropic regularization to a dual formulation of the permutahedron projection; see Section D.6 for details. This leads to a surrogate that behaves more like the entropic regularizer in the OT setting, which yields dense Jacobians. Moreover, by replacing the hard order-statistic bounds in the permutahedron LP with smooth log-sum-exp relaxations (Section D.6), this variant is $\mathcal{C}^\infty$ differentiable, unlike the other FastSoftSort variants which are at most $\mathcal{C}^2$. Unfortunately, we cannot use the same fast PAV algorithm in this

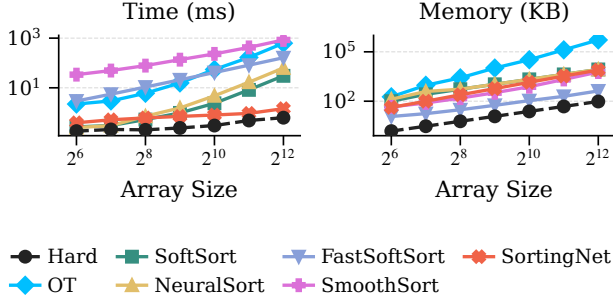


Figure 8. Computation time (left) and peak memory consumption (right) for a full forward-backward pass of `sj.sort` using `smooth` regularization. All six soft sorting methods are compared against the hard baseline. Extended results across all modes are in Figure 17.

case, because the smooth bounds require $O(n^2)$ preprocessing and the LP is solved via L-BFGS. However, compared to entropy-regularized OT, the $O(n^2)$ cost is paid only once (for the bounds), not per iteration, and we do not need to materialize an $n \times n$ matrix.

4.4. Sorting network-based

An alternative approach that avoids both the $O(n^2)$ pairwise cost matrix and iterative optimization is to use a differentiable sorting network. Following Petersen et al. (2021), who propose replacing the hard compare-and-swap operations in a bitonic sorting network with soft comparisons based on a logistic sigmoid, we implement a bitonic sorting network using our soft Heaviside function H_τ (Equations (5) and (6)) for soft swapping, i. e.,

$$\sigma = H_\tau(a - b), \quad (50)$$

$$\text{soft_min}(a, b) = \sigma \cdot b + (1 - \sigma) \cdot a, \quad (51)$$

$$\text{soft_max}(a, b) = \sigma \cdot a + (1 - \sigma) \cdot b. \quad (52)$$

In `smooth` mode, H_τ is the logistic sigmoid, directly recovering the method of Petersen et al. (2021), while `c0`, `c1`, and `c2` modes yield new sorting network variants with the corresponding smoothness guarantees. Following Petersen et al. (2021), we obtain soft permutation matrices by tracking an $n \times n$ matrix P (initialized to the identity) through the network. At each compare-and-swap step, the rows of P corresponding to the two compared positions are mixed using the weights σ and $1 - \sigma$, hence $\text{sort}(\mathbf{x}) = P\mathbf{x}$. Unlike FastSoftSort, sorting networks can therefore produce both sorted values and soft permutation matrices, enabling `argsort` and `argmax` in addition to `sort`, `max`, `min`, `quantile`, `median`, and `top_k`.

5. Benchmarks & Case Study

In order to be useful for machine learning practitioners, softened functions and their gradients must be both fast to compute and memory efficient. SoftJAX allows us to easily

compare these aspects across different methods within a unified framework. Figure 8 shows the computation time and peak memory of `sj.sort` for different input sizes and methods (`smooth` mode) on an Nvidia RTX 3060 GPU. The sorting network is the fastest soft method at 1.0 ms for $n = 4096$ (only $\sim 3.8\times$ the hard baseline), followed by SoftSort (16 ms) and NeuralSort (37 ms). FastSoftSort is the most memory-efficient method, scaling roughly linearly (420 kB at $n = 4096$) since it avoids materializing a permutation matrix. SmoothSort and the OT-based surrogate are the slowest of the tested methods. Extensive benchmarking results on all methods and modes for a variety of axis-wise functions are included in Appendix E, and Appendix F distills these results into practical guidance for choosing among methods and modes.

Additionally, Appendix B collects case studies that demonstrate the ease of using SoftJAX in practice: softening a subroutine of MuJoCo XLA collision detection, four-digit MNIST sorting and quantile regression, a differentiable 3D mesh renderer, and minimal use-case examples covering top- k feature selection and a differentiable rule-based classifier.

6. Conclusion

Standard tensor libraries with GPU acceleration and automatic differentiation have enabled differentiable optimization at scale in ML research. For many problems in ML and other fields, discrete primitives are needed, but result in uninformative gradients. Differentiable alternatives are scattered across papers, slowing progress and adoption. This work introduces SoftJAX and SoftTorch: feature-complete, unified and extensible libraries for soft relaxations of hard operators with principled straight-through estimation.

We systematically derive differentiable surrogates for elementwise discrete operators from smooth relaxation of the Heaviside step function. Our framework also provides a broad set of axiswise differentiable discrete operators including `argsort`, `argquantile`, and `argtop_k` – implemented via state-of-the-art algorithms based on optimal transport and simplex/permutahedron projections and derive index-selection primitives (e.g., `take_along_axis`, `choose`) for end-to-end use. Through this standardization, this work derives a broad range of differentiable operators for which an implementation is currently not easily available (see Table 2).

Overall, the framework consolidates core components for *soft differentiable programming* into a single, well-tested, feature-complete library, improving reproducibility and lowering the barrier to widespread use of soft differentiable programming in practice.

Acknowledgements

The authors thank Onur Beker for helpful discussions.

Additionally, we thank the International Max Planck Research School for Intelligent Systems (IMPRS-IS) for supporting Anselm Paulus.

The work on this paper was supported by the Czech Science Foundation (GAČR) grant no. 26-23981S.

This work was supported by the ERC - 101045454 REAL-RL and the German Federal Ministry of Education and Research (BMBF) through the Tübingen AI Center (FKZ: 01IS18039A). Georg Martius is a member of the Machine Learning Cluster of Excellence, EXC number 2064/1 – Project number 390727645.

Impact Statement

By consolidating soft relaxation techniques into open-source libraries for JAX and PyTorch, this work lowers the barrier to applying gradient-based optimization in domains where hard, non-differentiable operations have traditionally required specialized expertise or ad-hoc workarounds. We expect the primary impact to be positive, as practitioners in robotics, scientific simulation, and structured prediction can more easily incorporate informative gradients into their pipelines. As with any tool that broadens access to optimization techniques, the libraries could in principle be applied to objectives whose optimization raises ethical concerns, but this risk is not specific to our contribution and is shared by all automatic differentiation frameworks.

References

- Agrawal, A., Amos, B., Barratt, S. T., Boyd, S. P., Diamond, S., and Kolter, J. Z. [Differentiable convex optimization layers](#). In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pp. 9558–9570, 2019.
- Amos, B. and Kolter, J. Z. [Optnet: Differentiable optimization as a layer in neural networks](#). In *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017*, pp. 136–145, 2017.
- Amos, B. and Yarats, D. [The differentiable cross-entropy method](#). In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, pp. 291–302, 2020.
- Amos, B., Rodriguez, I. D. J., Sacks, J., Boots, B., and Kolter, J. Z. [Differentiable MPC for end-to-end planning and control](#). In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pp. 8299–8310, 2018.
- Beker, O., Gürtler, N., Shi, J., Geist, A. R., Razmjoo, A., Martius, G., and Calinon, S. [A smooth analytical formulation of collision detection and rigid body dynamics with contact](#). In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 18558–18565, 2025. doi: 10.1109/IROS60139.2025.11247449.
- Beker, O., Geist, A. R., Paulus, A., Gürtler, N., Shi, J., Calinon, S., and Martius, G. [Smoothly differentiable and efficiently vectorizable contact manifold generation](#). *arXiv:2602.20304*, 2026.
- Belohlavek, R., Dauben, J. W., and Klir, G. J. *Fuzzy logic and mathematics: a historical perspective*. Oxford University Press, 2017.
- Bengio, Y., Léonard, N., and Courville, A. C. [Estimating or propagating gradients through stochastic neurons for conditional computation](#). *CoRR*, abs/1308.3432, 2013.
- Berthet, Q., Blondel, M., Teboul, O., Cuturi, M., Vert, J., and Bach, F. R. [Learning with differentiable perturbed optimizers](#). In *Advances in Neural Information Processing Systems*, volume 33, pp. 9508–9519, 2020.
- Blondel, M., Seguy, V., and Rolet, A. [Smooth and sparse optimal transport](#). In *International Conference on Artificial Intelligence and Statistics, AISTATS 2018, 9-11 April 2018, Playa Blanca, Lanzarote, Canary Islands, Spain*, pp. 880–889, 2018.
- Blondel, M., Teboul, O., Berthet, Q., and Djolonga, J. [Fast differentiable sorting and ranking](#). In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, pp. 950–959, 2020.
- Blondel, M., Berthet, Q., Cuturi, M., Frostig, R., Hoyer, S., Llinares-López, F., Pedregosa, F., and Vert, J. [Efficient and modular implicit differentiation](#). In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Katariya, Y., Leary, C., Maclaurin, D., Necula, G., Paszke, A., Vanderplas, J., Wanderman-Milne, S., and Zhang, Q. [JAX: composable transformations of Python+NumPy programs](#), 2018.

- Bregman, L. [The relaxation method of finding the common point of convex sets and its application to the solution of problems in convex programming](#). *USSR Computational Mathematics and Mathematical Physics*, 7(3):200–217, 1967. ISSN 0041-5553. doi: [https://doi.org/10.1016/0041-5553\(67\)90040-7](https://doi.org/10.1016/0041-5553(67)90040-7).
- Breiman, L., Friedman, J., Stone, C. J., and Olshen, R. A. *Classification and Regression Trees*. Chapman & Hall/CRC, January 1984.
- Cuturi, M. [Sinkhorn distances: Lightspeed computation of optimal transport](#). In *Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States*, pp. 2292–2300, 2013.
- Cuturi, M., Teboul, O., and Vert, J.-P. [Differentiable ranking and sorting using optimal transport](#). In Wallach, H., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- Cuturi, M., Meng-Papaxanthos, L., Tian, Y., Bunne, C., Davis, G., and Teboul, O. [Optimal transport tools \(ott\): A jax toolbox for all things wasserstein](#). *arXiv preprint arXiv:2201.12324*, 2022.
- Duch, W. and Jankowski, N. [Survey of neural transfer functions](#). *Neural computing surveys*, 2(1):163–212, 1999.
- Flamary, R., Courty, N., Gramfort, A., Alaya, M. Z., Boissunon, A., Chambon, S., Chapel, L., Corenflos, A., Fatras, K., Fournier, N., Gautheron, L., Gayraud, N. T., Janati, H., Rakotomamonjy, A., Redko, I., Rolet, A., Schutz, A., Seguy, V., Sutherland, D. J., Tavenard, R., Tong, A., and Vayer, T. [POT: Python optimal transport](#). *Journal of Machine Learning Research*, 22(78):1–8, 2021.
- Flamary, R., Vincent-Cuaz, C., Courty, N., Gramfort, A., Kachaiev, O., Quang Tran, H., David, L., Bonet, C., Cassereau, N., Gnassounou, T., Tanguy, E., Delon, J., Collas, A., Mazelet, S., Chapel, L., Kerdoncuff, T., Yu, X., Feickert, M., Krzakala, P., Liu, T., and Fernandes Montesuma, E. [Pot python optimal transport \(version 0.9.5\)](#), 2024.
- Funahashi, K.-I. [On the approximate realization of continuous mappings by neural networks](#). *Neural networks*, 2(3): 183–192, 1989.
- Geist, A. R., Frey, J., Zhobro, M., Levina, A., and Martius, G. Learning with 3D rotations, a hitchhiker’s guide to $SO(3)$. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 15331–15350. PMLR, 21–27 Jul 2024.
- Grover, A., Wang, E., Zweig, A., and Ermon, S. [Stochastic optimization of sorting networks via continuous relaxations](#). In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*, 2019.
- Guyon, I. and Elisseeff, A. An introduction to variable and feature selection. *J. Mach. Learn. Res.*, 3(null): 1157–1182, March 2003. ISSN 1532-4435.
- Han, J. and Moraga, C. [The influence of the sigmoid function parameters on the speed of backpropagation learning](#). In *International workshop on artificial neural networks*, pp. 195–201. Springer, 1995.
- Hendrycks, D. and Gimpel, K. [Bridging nonlinearities and stochastic regularizers with gaussian error linear units](#). *CoRR*, abs/1606.08415, 2016.
- Howell, T., Google DeepMind, and NVIDIA. [MuJoCo Warp \(MJWarp\)](#), 2025.
- Huang, G.-B., Chen, L., and Siew, C.-K. [Universal approximation using incremental constructive feedforward networks with random hidden nodes](#). *IEEE Transactions on Neural Networks*, 17(4):879–892, 2006a. doi: 10.1109/TNN.2006.875977.
- Huang, G.-B., Zhu, Q.-Y., Mao, K., Siew, C.-K., Saratchandran, P., and Sundararajan, N. [Can threshold networks be trained directly?](#) *IEEE Transactions on Circuits and Systems II: Express Briefs*, 53(3):187–191, 2006b.
- Jang, E., Gu, S., and Poole, B. [Categorical reparameterization with gumbel-softmax](#). In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*, 2017.
- Kerbl, B., Kopanas, G., Leimkühler, T., and Drettakis, G. [3d gaussian splatting for real-time radiance field rendering](#). *ACM Trans. Graph.*, 42(4):139:1–139:14, 2023. doi: 10.1145/3592433.
- Kidger, P. and Garcia, C. [Equinox: neural networks in JAX via callable PyTrees and filtered transformations](#). *Differentiable Programming workshop at Neural Information Processing Systems 2021*, 2021.
- LeCun, Y., Cortes, C., and Burges, C. [Mnist handwritten digit database](#). 2010.
- Levinson, J., Esteves, C., Chen, K., Snively, N., Kanazawa, A., Rostamizadeh, A., and Makadia, A. An analysis of svd for deep rotation estimation. *Advances in Neural Information Processing Systems*, 33:22554–22565, 2020.

- Li, Y., Guo, Y., Zhang, S., Deng, S., Hai, Y., and Gu, S. [Differentiable spike: Rethinking gradient-descent for training spiking neural networks](#). In Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems*, volume 34, pp. 23426–23439. Curran Associates, Inc., 2021.
- Liu, S., Li, T., Chen, W., and Li, H. Soft rasterizer: A differentiable renderer for image-based 3d reasoning. *The IEEE International Conference on Computer Vision (ICCV)*, Oct 2019.
- Lu, L., Shin, Y., Su, Y., and Em Karniadakis, G. [Dy-ing relu and initialization: Theory and numerical examples](#). *Communications in Computational Physics*, 28(5):1671–1706, June 2020. ISSN 1991-7120. doi: 10.4208/cicp.oa-2020-0165.
- Maddison, C. J., Mnih, A., and Teh, Y. W. [The concrete distribution: A continuous relaxation of discrete random variables](#). In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*, 2017.
- Martins, A. F. T. and Astudillo, R. F. [From softmax to sparsemax: A sparse model of attention and multi-label classification](#). In *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, pp. 1614–1623, 2016.
- Mhaskar, H. N. Neural networks for optimal approximation of smooth and analytic functions. *Neural Computation*, 8(1):164–177, 1996. doi: 10.1162/neco.1996.8.1.164.
- Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., and Ng, R. [Nerf: Representing scenes as neural radiance fields for view synthesis](#). In *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part I*, pp. 405–421, 2020. doi: 10.1007/978-3-030-58452-8_24.
- MuJoCo, T. [MuJoCo XLA \(MJX\)](#), 2024.
- Niepert, M., Minervini, P., and Franceschi, L. [Implicit MLE: backpropagating through discrete exponential family distributions](#). In *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pp. 14567–14579, 2021.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. [Pytorch: An imperative style, high-performance deep learning library](#). In *Advances in Neural Information Processing Systems 32*, pp. 8024–8035. Curran Associates, Inc., 2019.
- Paty, F. and Cuturi, M. [Regularized optimal transport is ground cost adversarial](#). In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, pp. 7532–7542, 2020.
- Paulus, A., Martius, G., and Musil, V. [LPGD: A general framework for backpropagation through embedded optimization layers](#). In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, 2024.
- Paulus, A., Geist, A. R., Schumacher, P., Musil, V., and Martius, G. [Differentiable simulation of hard contacts with soft gradients for learning and control](#). In *International Conference on Learning Representations*, 2026.
- Paulus, M. B., Choi, D., Tarlow, D., Krause, A., and Maddison, C. J. [Gradient estimation with stochastic softmax tricks](#). In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.
- Petersen, F., Borgelt, C., Kuehne, H., and Deussen, O. [Differentiable sorting networks for scalable sorting and ranking supervision](#). In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, pp. 8546–8555, 2021.
- Petersen, F., Borgelt, C., Kuehne, H., and Deussen, O. Monotonic differentiable sorting networks. In *International Conference on Learning Representations (ICLR)*, 2022a.
- Petersen, F., Goldlücke, B., Borgelt, C., and Deußen, O. [Gendr: A generalized differentiable renderer](#). *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3992–4001, 2022b. doi: 10.1109/cvpr52688.2022.00397.
- Prillo, S. and Eisenschlos, J. M. [Softsort: A continuous relaxation for the argsort operator](#). In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, pp. 7793–7802, 2020.
- Rader, J., Lyons, T., and Kidger, P. [Optimistix: modular optimisation in jax and equinox](#). *arXiv:2402.09983*, 2024.
- Rojas, R. [The backpropagation algorithm](#). In *Neural Networks: A Systematic Introduction*, pp. 149–182. Springer Berlin Heidelberg, Berlin, Heidelberg, 1996. ISBN 978-3-642-61068-4. doi: 10.1007/978-3-642-61068-4_7.

- Rolínek, M., Musil, V., Paulus, A., P., M. V., Michaelis, C., and Martius, G. [Optimizing rank-based metrics with blackbox differentiation](#). In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*, pp. 7617–7627, 2020a. doi: 10.1109/CVPR42600.2020.00764.
- Rolínek, M., Swoboda, P., Zietlow, D., Paulus, A., Musil, V., and Martius, G. [Deep graph matching via blackbox differentiation of combinatorial solvers](#). In *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part XXVIII*, pp. 407–424, 2020b. doi: 10.1007/978-3-030-58604-1_25.
- Rosenblatt, F. [The perceptron: A probabilistic model for information storage and organization in the brain](#). *Psychological Review*, 65(6):386–408, 1958. doi: 10.1037/h0042519.
- Ruiz-Gonzalez, C., Arlt, S., Lehner, S., Berzins, A., Drori, Y., Adhikari, R. X., Brandstetter, J., and Krenn, M. [Neural surrogates for designing gravitational wave detectors](#), 2025.
- Sahoo, S. S., Paulus, A., Vlastelica, M., Musil, V., Kuleshov, V., and Martius, G. [Backpropagation through combinatorial algorithms: Identity with projection works](#). In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*, 2023.
- Sander, M. E., Puigcerver, J., Djolonga, J., Peyré, G., and Blondel, M. [Fast, differentiable and sparse top-k: a convex analysis perspective](#). In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, pp. 29919–29936, 2023.
- Semenov, S. [Smooth approximations of the rounding function](#). *CoRR*, abs/2504.19026, 2025. doi: 10.48550/ARXIV.2504.19026.
- Stumm, P. and Walther, A. [New algorithms for optimal on-line checkpointing](#). *SIAM J. Sci. Comput.*, 32(2):836–854, March 2010. ISSN 1064-8275.
- Todorov, E., Erez, T., and Tassa, Y. [Mujoco: A physics engine for model-based control](#). In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033. IEEE, 2012. doi: 10.1109/IROS.2012.6386109.
- Vlastelica, M., Paulus, A., Musil, V., Martius, G., and Rolínek, M. [Differentiation of blackbox combinatorial solvers](#). In *International Conference on Learning Representations*, 2020.
- Wang, W. and Carreira-Perpiñán, M. Á. [Projection onto the probability simplex: An efficient algorithm with a simple proof, and an application](#). *CoRR*, abs/1309.1541, 2013.
- Wang, Y., Serena, F., Wu, S., Öztireli, C., and Sorkine-Hornung, O. [Differentiable surface splatting for point-based geometry processing](#). *ACM Trans. Graph.*, 38(6): 230:1–230:14, 2019. doi: 10.1145/3355089.3356513.
- Xie, Y., Dai, H., Chen, M., Dai, B., Zhao, T., Zha, H., Wei, W., and Pfister, T. [Differentiable top-k with optimal transport](#). In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 20520–20531. Curran Associates, Inc., 2020.
- Yuen, B., Hoang, M. T., Dong, X., and Lu, T. [Universal activation function for machine learning](#). *Scientific reports*, 11(1):18757, 2021.

A. Library Overview

Axiswise	Elementwise	Logical	Selection	Modes
argmin	heaviside	logical_and	where	✓ smooth
min	round	logical_or	take_along_axis	✓ c0
argsort	sign	logical_xor	take	✓ c1
sort	abs	logical_not	choose	✓ c2
argquantile	relu	any	dynamic_index_in_dim	Methods ✓ Optimal Transport ✓ SoftSort ✓ NeuralSort ✓ FastSoftSort ✓ SmoothSort ✓ Sorting Network
quantile	clip	all	dynamic_slice_in_dim	
argmedian	less		dynamic_slice	
median	less_equal			
top_k	equal			
rank	not_equal			
	isclose			

Figure 9. Overview of all operators implemented in SoftJAX. Redundant operators such as max or greater have been omitted for brevity. Additionally, SoftJAX provides autograd-safe wrappers for common mathematical functions whose standard implementations produce NaN gradients at boundary points: sqrt, arcsin, arccos, div, log, and norm. These wrappers clamp gradients near singularities, making them safe for use in differentiable pipelines. SoftTorch provides the same operators using PyTorch naming conventions; see Table 1 for the mapping.

Table 1. Naming differences between SoftJAX and SoftTorch. SoftTorch follows PyTorch conventions. Functions listed as “—” are not available in SoftTorch (no PyTorch equivalent).

SoftJAX	SoftTorch
clip	clamp
equal	eq
top_k	topk
take_along_axis	take_along_dim
dynamic_index_in_dim	index_select
dynamic_slice_in_dim	narrow
choose	—
dynamic_slice	—

Table 2. Overview of algorithms used. While not all regularization methods are detailed in the literature for the various algorithm types, we derived the missing combinations by systematically combining the above-listed works.

	Optimal transport		
	smooth	c0 (Blondel et al., 2018)	c1/c2 (Paty & Cuturi, 2020)
argmax	(Cuturi et al., 2019)	derived	derived
argsort	(Cuturi et al., 2019)	derived	derived
argquantile	(Cuturi et al., 2019)	derived	derived
argmedian	(Cuturi et al., 2019)	derived	derived
argtop_k	(Xie et al., 2020)	derived	derived
rank	(Cuturi et al., 2019)	derived	derived
	SoftSort		
	smooth	c0	c1/c2
argmax	softmax	(Martins & Astudillo, 2016)	derived
argsort	(Prillo & Eisenschlos, 2020)	derived	derived
argquantile	(Prillo & Eisenschlos, 2020)	derived	derived
argmedian	(Prillo & Eisenschlos, 2020)	derived	derived
argtop_k	(Prillo & Eisenschlos, 2020)	derived	derived
rank	derived	derived	derived
	NeuralSort		
	smooth	c0	c1/c2
argmax	(Grover et al., 2019)	derived	derived
argsort	(Grover et al., 2019)	derived	derived
argquantile	(Grover et al., 2019)	derived	derived
argmedian	(Grover et al., 2019)	derived	derived
argtop_k	(Grover et al., 2019)	derived	derived
rank	derived	derived	derived
	FastSoftSort		
	smooth	c0	c1/c2
sort	(Blondel et al., 2020)	(Blondel et al., 2020)	(Sander et al., 2023)
rank	(Blondel et al., 2020)	(Blondel et al., 2020)	(Sander et al., 2023)
	SmoothSort		
	smooth	c0	c1/c2
sort	novel	—	—
rank	novel	—	—
	Sorting network		
	smooth	c0	c1/c2
sort	(Petersen et al., 2021)	derived	derived
argsort	(Petersen et al., 2021)	derived	derived

B. Case Studies

B.1. Collision detection in MuJoCo XLA

Mujoco (Todorov et al., 2012) is one of the most widely used simulators throughout robotics research. MuJoCo XLA (MJX) is a re-implementation of MuJoCo using JAX to enable GPU-parallelized automatic differentiation of robot simulations. Unfortunately, neither MuJoCo nor MJX are softly differentiable as emphasized by Paulus et al. (2026). To improve differentiability, Paulus et al. (2026) introduced soft relaxations of MuJoCo’s collision detection algorithms for simple primitives, i. e., sphere-plane and box-plane collisions. However, softening MuJoCo’s collision detection for mesh-mesh collisions is significantly more challenging as it involves numerous nested combinations of non-differentiable discrete operations. Alternatively, Beker et al. (2025; 2026) propose efficient soft collision detection via signed distance fields. To illustrate the utility of SoftJAX, we soften one of the key subroutines in MJX collision detection. The subroutine as shown in Figure 12 (left) chooses four vertices (A, B, C, D) from an ordered polygon that roughly maximize the contact patch area.

To render this algorithm smoothly differentiable, we convert it to SoftJax as shown in Figure 12 (right). For this, the

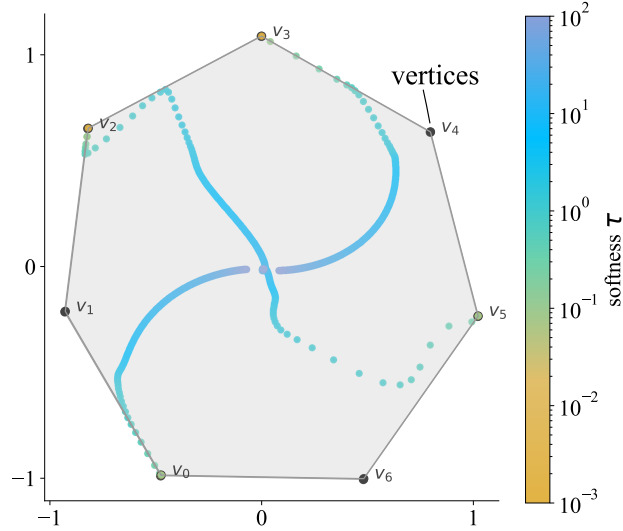


Figure 10. Manifold points computed by the algorithm’s SoftJAX reimplementation over different softness values τ . While a small softness causes the algorithm to choose points close to the vertices of the polygon, for large softness values, all four points collapse into a single point.

following changes are made:

1. Replace discrete `jnp.argmax` with `sj.argmax`, which returns a “SoftIndex” distribution,
2. Replace indexing with `sj.dynamic_index_in_dim`, which uses the “SoftIndex” as input,
3. Replace `jnp.abs` with `sj.abs`.

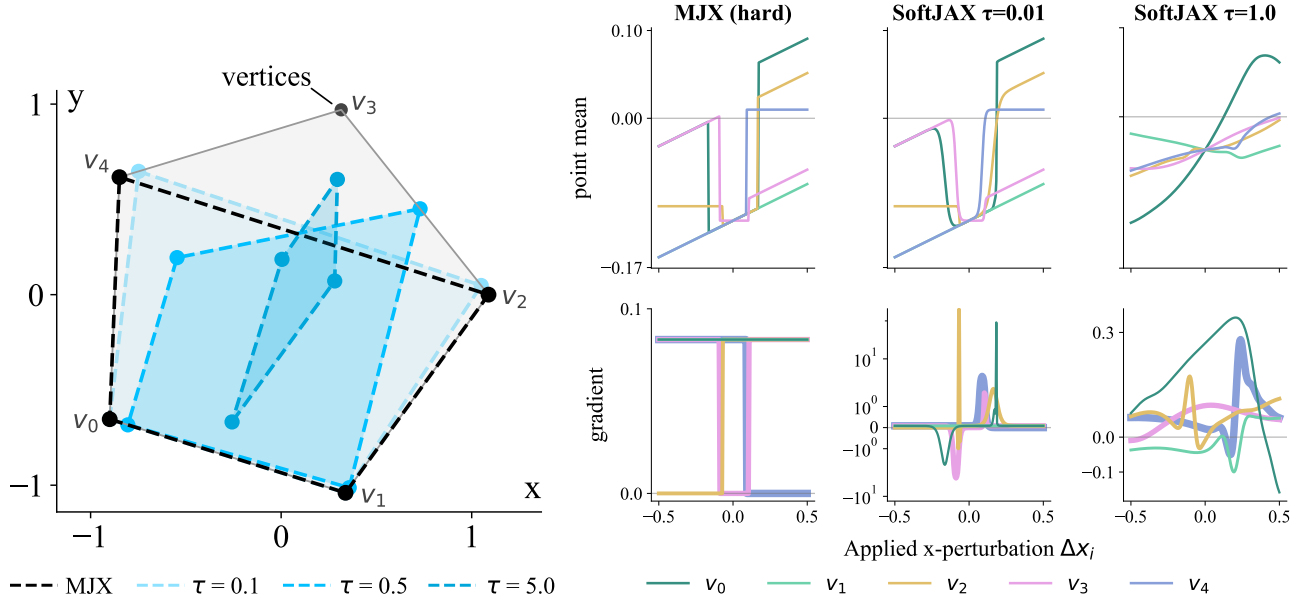
Note that the soft version returns “SoftIndex” distributions (shape: $4 \times n$) instead of the hard indices. The remaining code of the collision detection, therefore, needs to be adjusted accordingly.

Effect of smoothing on point selection and gradients. Figure 10 illustrates how the soft algorithm chooses four points from a seven-sided polygon as the softness parameter varies (computed as `soft_probs @ poly`). For small softness values, the soft selection closely matches the hard algorithm. As softness increases, the selected points meet in a single point as the `sj.argmax` reduces to the computation of a mean, as also shown in Figure 11 (Left).

A key motivation for softening is to obtain informative gradients with respect to all function inputs. As shown in Figure 11 (Right), MJX’s hard version of manifold point selection always chooses four points out of the polygon’s five vertices such that for one polygon point, the gradient is null. In comparison, the algorithm’s soft re-implementation yields smooth, non-zero gradients at all vertices of the polygon (here deploying optimal transport with `smooth` regularization). Small softness values (e. g., $\tau = 0.01$) cause the algorithm to select points similarly to its hard counterpart at the expense of increased gradient magnitudes (note the logarithmic y-axis). When small softness values are required in an application, we recommend to use gradient clipping to improve numerical stability.

Avoiding the STE pitfall. We could now directly use one of the relaxations as a differentiable proxy. However, oftentimes it is desirable not to alter the forward pass. For instance, in a physics simulation, we do not want to alter the forward physics. In these cases, we can resort to the straight-through trick, which means to replace only the gradient of a hard function on the forward pass with the gradient of the soft function on the backward pass. To avoid the STE pitfall, the straight-through trick is applied on the outer level of the downstream function, which calls the whole function twice instead of each of the primitives:

```
from softjax.straight_through import st
manifold_points_softjax_st = st(manifold_points_softjax)
```



```
def manifold_points_mjx(poly, poly_mask, poly_norm:
    jnp.ndarray):

    dist_mask = jnp.where(poly_mask, 0.0, -1e6)
    # Note: We add a small tie-breaker
    # A: select the most penetrating vertex
    a_logits = dist_mask - 0.1 * jnp.arange(poly.shape
        [0])
    a_idx = jnp.argmax(a_logits)
    a = poly[a_idx]

    # B: farthest from A (largest squared distance)
    b_logits = ((a - poly) ** 2).sum(axis=1) +
        dist_mask
    b_idx = jnp.argmax(b_logits)
    b = poly[b_idx]

    # C: farthest from the AB line within the plane
    ab = jnp.cross(poly_norm, a - b)
    ap = a - poly
    c_logits = jnp.abs(ap.dot(ab)) + dist_mask
    c_idx = jnp.argmax(c_logits)
    c = poly[c_idx]

    # D: farthest from edges AC and BC
    ac = jnp.cross(poly_norm, a - c)
    bc = jnp.cross(poly_norm, b - c)
    bp = b - poly
    dist_bp = jnp.abs(bp.dot(bc)) + dist_mask
    dist_ap = jnp.abs(ap.dot(ac)) + dist_mask
    d_logits = dist_bp + dist_ap
    d_idx = jnp.argmax(d_logits)
    return jnp.array([a_idx, b_idx, c_idx, d_idx])
```

```
def manifold_points_softjax(poly, poly_mask, poly_norm
    , softness: float = 0.1, mode: Literal["hard", "
    smooth"] = "smooth"):
    m, s = mode, softness
    dist_mask = jnp.where(poly_mask, 0.0, -1e6)

    # A: soft argmax over masked distances
    a_logits = dist_mask - 0.1 * jnp.arange(poly.shape
        [0])
    a_idx = sj.argmax(a_logits, mode=m, softness=s)
    a = sj.dynamic_index_in_dim(poly, a_idx, axis=0,
        keepdims=False)

    # B: soft argmax of distance from A
    b_logits = (((a - poly) ** 2).sum(axis=1)) +
        dist_mask
    b_idx = sj.argmax(b_logits, mode=m, softness=s)
    b = sj.dynamic_index_in_dim(poly, b_idx, axis=0,
        keepdims=False)

    # C: soft argmax farthest from AB line
    ab = jnp.cross(poly_norm, a - b)
    ap = a - poly
    c_logits = sj.abs(ap.dot(ab), mode=m, softness=s)
        + dist_mask
    c_idx = sj.argmax(c_logits, mode=m, softness=s)
    c = sj.dynamic_index_in_dim(poly, c_idx, axis=0,
        keepdims=False)

    # D: softargmax farthest from AC and BC
    ac = jnp.cross(poly_norm, a - c)
    bc = jnp.cross(poly_norm, b - c)
    bp = b - poly
    dist_bp = sj.abs(bp.dot(bc), mode=m, softness=s) +
        dist_mask
    dist_ap = sj.abs(ap.dot(ac), mode=m, softness=s) +
        dist_mask
    d_logits = dist_bp + dist_ap
    d_idx = sj.argmax(d_logits, mode=m, softness=s)
    return jnp.stack([a_idx, b_idx, c_idx, d_idx])
```

Figure 12. Left: Non-differentiable algorithm used by MJX for collision detection between mesh-mesh collisions. The algorithm chooses four vertices from a polygon that form a quadrilateral with approximately largest area. Right: Softened version of MJX’s manifold point algorithm yielding informative gradients and returning four “SoftIndex” distributions.

B.2. Sorting and quantile regression of four-digit MNIST

Sequences of four-digit MNIST numbers have been commonly used in literature to validate soft relaxations of sorting and quantile computation, see e. g., (Grover et al., 2019; Cuturi et al., 2019; Blondel et al., 2020; Petersen et al., 2021; 2022a). Following these works, each data sample contains a sequence of n images depicting four-digit numbers ranging from 0000 to 9999, each of which, consists of four MNIST images (LeCun et al., 2010) stacked along the width dimension (yielding images of size $1 \times 28 \times 112$). For each four-digit image in the sequence, a CNN predicts a scalar, yielding the sequence of scalars $a_1, \dots, a_n$. Depending on the task at hand the scalars are either fed to a differentiable argsort or quantile operator whose output is used to train the CNN. The CNN architecture consists of two convolutional layers (32 and 64 filters of size 5×5 with padding 2, each followed by ReLU and 2×2 max-pooling), a fully connected layer with 64 hidden units and ReLU, and a linear output layer producing a single scalar. Training samples are generated on the fly by randomly drawing with replacement from a pool of 55,000 MNIST training digits to form the sequence of four-digit numbers; the remaining 5,000 digits form the validation set and 10,000 test digits are used for evaluation. We train for 100,000 gradient steps using Adam with a learning rate of 0.001 and a batch size of 100.

Sorting. In this experiment, the sequence of scalars $a_1, \dots, a_n$ is fed to SoftJAX’s soft argsort operators. The argsort operation outputs a permutation matrix P which is compared in a cross-entropy objective to the ground-truth permutation matrix Q – defining the correct sorting operation for the four-digit sequence of numbers. Our reimplementation of this experiment is based on the code of (Petersen et al., 2022a) which has been ported to JAX using Equinox (Kidger & Garcia, 2021). Table 3 reports exact-match accuracy (proportion of sequences whose predicted ranking is entirely correct) and element-wise accuracy (proportion of individual positions ranked correctly).

Quantile regression. Following (Prillo & Eisenschlos, 2020), instead of predicting the full ranking, the CNN outputs are fed to SoftJAX’s quantile operator and is trained to predict the q -quantile (e.g., median for $q = 0.5$) of the scores. The soft quantile operator extracts a differentiable quantile value from the CNN scores, trained via mean squared error against the true quantile of the numerical labels. At evaluation time, the hard quantile is used. Table 4 reports MSE and in brackets the Spearman rank correlation between predicted and true quantile values across the test set.

Table 3. Results of training a CNN to sort four-digit MNIST sequences of length n . Reported metric is the percentage of correctly classified sequences while the brackets report the proportion of correctly identified individual element ranks.

smooth ($\tau = 0.1$)	$n = 3$	$n = 5$	$n=7$	$n=9$	$n=15$	$n=32$
SoftSort	93.0 (95.3)	82.6 (92.2)	69.5 (89.2)	58.8 (87.3)	24.0 (79.1)	0.2 (62.1)
NeuralSort	92.5 (94.9)	81.7 (91.8)	69.8 (89.4)	58.6 (87.4)	25.5 (79.7)	0.0 (37.1)
Sorting Network	92.6 (94.9)	81.6 (91.8)	67.5 (88.5)	48.5 (84.0)	16.6 (76.3)	0.0 (43.4)
OT	92.7 (95.0)	82.1 (92.0)	68.9 (88.9)	49.9 (84.3)	6.0 (66.1)	0.0 (23.6)
c2 ($\tau = 0.1$)	$n=3$	$n=5$	$n=7$	$n=9$	$n=15$	$n=32$
SoftSort	88.0 (91.7)	80.6 (91.5)	66.4 (88.1)	56.2 (86.8)	30.2 (81.7)	2.2 (70.7)
NeuralSort	90.0 (93.2)	78.2 (90.3)	63.4 (87.1)	49.2 (84.2)	8.5 (69.7)	0.0 (52.8)
Sorting Network	90.3 (93.4)	79.7 (90.9)	65.2 (87.7)	45.9 (82.9)	13.1 (74.0)	0.0 (35.4)
OT	87.6 (91.5)	84.1 (93.0)	73.2 (90.7)	58.0 (87.2)	27.1 (80.7)	1.3 (68.0)
c0 ($\tau = 0.1$)	$n=3$	$n=5$	$n=7$	$n=9$	$n=15$	$n=32$
SoftSort	59.1 (70.9)	16.4 (50.6)	4.8 (44.2)	0.8 (37.3)	0.0 (25.5)	0.0 (16.6)
NeuralSort	63.2 (73.9)	14.5 (48.9)	0.7 (29.0)	0.04 (21.9)	0.0 (11.4)	0.0 (5.3)
Sorting Network	74.9 (82.5)	68.6 (85.8)	49.1 (80.7)	4.2 (51.3)	0.1 (42.0)	0.0 (14.4)
OT	76.8 (83.9)	53.1 (77.5)	28.5 (69.6)	11.0 (61.0)	0.0 (32.8)	0.0 (10.6)

Table 4. Results of training a CNN to predict the 0.5-quantile of four-digit MNIST sequences of length n . Reported metric is the MSE ($\times 10^{-4}$) when predicting the quantile, and in brackets, the Spearman rank correlation between predicted and true quantile values.

smooth ($\tau = 0.1$)	$n=3$	$n=5$	$n=7$	$n=9$	$n=15$
SoftSort	30.9 (97.4)	26.8 (96.9)	21.4 (96.9)	21.4 (96.4)	15.7 (95.8)
NeuralSort	26.8 (97.7)	18.7 (97.8)	24.2 (96.1)	24.4 (95.4)	16.3 (95.1)
Sorting Network	31.9 (97.2)	22.8 (97.6)	25.1 (96.8)	21.2 (96.8)	26.4 (93.6)

B.3. Differentiable rendering

To illustrate the merits of SoftJAX and SoftTorch for quick prototyping, we implement a differentiable 3D mesh renderer in JAX. The implementation is based on Sebastian Lague’s “Coding Adventure: Software Rasterizer” whose code is first ported to JAX and then extended to use soft operators drawing inspiration from Liu et al. (2019). This example is solely for the purpose of illustration, for professional differentiable rendering software, the interested reader is referred to Mitsuba 3.

Rasterization. Given a triangle mesh M and extrinsic variables (camera and lighting conditions), the vertices of each triangle are first projected onto the 2D image plane. For each projected triangle $f_j = \{a, b, c\}$, it is then determined for each 2D image pixel location l_i , if it lies inside the triangle. The probability p_j^i indicating if l_i lies inside f_j is computed using Barycentric coordinates b_a, b_b, b_c as

$$p_j^i = \text{all} (H_{\tau_1}(b_a), H_{\tau_1}(b_b), H_{\tau_1}(b_c)) . \quad (53)$$

where H_{τ_1} denotes the soft Bool operator with softness τ_1 . The barycentric coordinate b_a corresponds to the half signed area spanned by $c - b$ and $l_i - b$. In 2D, the signed area corresponds to the determinant of the matrix formed by $c - b$ and $l_i - b$ divided by two. In turn, given an image of size $H \times W$ and N triangles, (53) gives rise to an array P of size $N \times H \times W$ containing probabilities p_j^i . Further, the depth and color of all pixels relative to a given triangle are computed using clipped barycentric coordinates, giving rise to arrays of size $1 \times H \times W \times N$ and $3 \times H \times W \times N$.

Z-buffering. As next step, it is determined which of the triangles is close to the image pixels. To do so, following (Liu et al., 2019), the pixel-vertex-wise depth values d_j^i are transformed to inverse depth z_j^i such that 1 corresponds to the nearest vertex and 0 to the vertex that is furthest away from the respective pixel. In turn, a weight for each pixel-vertex pair is obtained through a weighted soft-argmax as

$$w_j^i = \frac{p_j^i \exp(z_j^i / \tau_2)}{\sum_k p_k^i \exp(z_k^i / \tau_2)} , \quad (54)$$

which we implemented using argmax_{τ_2} and noting that $b \exp(a) = \exp(a + \log(b))$ if $b > 0$. Note that the background was incorporated into (54) by appending an array of ones of size $1 \times H \times W$ to P as well as appending an array of size $1 \times H \times W$ containing a small number ϵ to the arrays of inverse depths. Subsequently, given w_j^i , the color of each image pixel is obtained as $I^i = \sum_j w_j^i C_j^i$. Note that the color aggregation operation equals the computation of an expectation using the `take_along_axis` operator.

Object pose estimation. Figure 14 shows how our differentiable renderer optimizes object poses to minimize the L2 image loss between a “hard” target image and the “softly” rendered image estimate. The pose consists of the object’s translational position and a nine dimensional rotation matrix representation based on the SVD decomposition as proposed by (Levinson et al., 2020) and discussed in detail in (Geist et al., 2024).

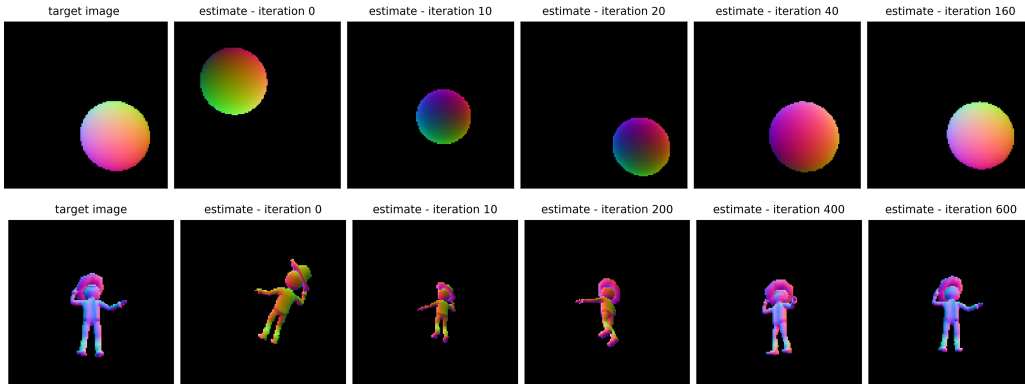


Figure 14. Gradient-based optimization of 3D object poses by minimizing the L2 errors between the hard target image and a softly rasterized image estimate. The objects are colored using a surface normal shader. Optimization uses AdamW ($\beta_1 = 0.9, \beta_2 = 0.99$) with stepsize 0.1 and softness parameters $\tau_1 = 0.1, \tau_2 = 0.1$. While the images show convergence for both objects, the loss landscape is highly non-linear and many initial positions did not converge. **Top:** Sphere object. **Bottom:** “Dave” figurine.

B.4. Minimal use-case examples

In this section, we provide two minimal and self-contained examples that showcase how SoftJAX can be applied in practice.

Top-k feature selection. A central challenge in applied machine learning is identifying the input features that most strongly influence model predictions (Guyon & Elisseeff, 2003). Given a trained model $f_\theta(x_1, \dots, x_n)$, we seek a size- k subset of features such that predictions are approximately preserved when all remaining features are masked. Formally, we seek the optimal index set

$$I^* = \arg \min_{\substack{I \subseteq \{1, \dots, n\} \\ |I|=k}} \mathbb{E}_{x_1, \dots, x_n \sim P} |f_\theta(x_1, \dots, x_n) - f_\theta(\hat{x}_1, \dots, \hat{x}_n)|, \quad (55)$$

where P denotes the data distribution and

$$\hat{x}_i = \begin{cases} x_i & \text{if } i \in I \\ 0 & \text{otherwise} \end{cases} \quad (56)$$

denotes the masked feature vector.

Exact solution requires an exhaustive search over all $\binom{n}{k}$ subsets, which is computationally demanding. Instead, we assign a scalar *importance score* $g_i \in \mathbb{R}$ to each feature and optimize these scores via gradient descent. To this end, we use `sj.top_k` to obtain a differentiable soft mask over the top- k scoring features. We evaluate on a linear model with sparse weights $w \in \mathbb{R}^{10}$ (only features 2, 4, 8 are non-zero). Gaussian noise is added to the targets to make the task more challenging and test robustness of the gradient signal. As shown in Figure 15(a), the smooth variant recovers the correct features in just a single step and preserves their ranking.

```
n_features, k = 10, 3
k1, k2 = jax.random.split(jax.random.PRNGKey(42))
X = jax.random.normal(k1, (100, n_features))
w_model = jnp.array([0, 2.0, 0, -1.5, 0, 0, 0, 5.0, 0, 0])
y = X @ w_model + 0.1 * jax.random.normal(k2, (100,))

def feature_selection_loss(g, X, y, w_model, mode="smooth"):
    _, soft_idx = sj.top_k(g, k=k, mode=mode, gated_grad=False)
    mask = soft_idx.sum(axis=0)
    y_pred = (X * mask) @ w_model
    return jnp.mean(sj.abs(y_pred - y))

g = jnp.zeros(n_features)
print("Hard grad:", jax.grad(feature_selection_loss)(g, X, y, w_model, mode="hard"))
print("Soft grad:", jax.grad(feature_selection_loss)(g, X, y, w_model, mode="smooth"))

for _ in range(5):
    g = g - 0.001 * jax.grad(feature_selection_loss)(g, X, y, w_model)
print("Selected features:", jax.lax.top_k(g, k=k)[1] + 1)
```

Output:

```
Hard grad: [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
Soft grad: [2268.416 -2371.1378 2268.416 1126.1998 2268.416 2268.416 2268.416 -14633.9742 2268.416 2268.416]
Selected features: [8 2 4]
```

Differentiable rule-based classifier. We demonstrate the utility of SoftJAX’s logical and inequality operators by learning a simple rule-based classifier via gradient descent. Specifically, we aim to fit the model

$$f_{l,h}(\mathbf{x}) = (l < \mathbf{x}_1 < h) \vee (l < \mathbf{x}_2 < h), \quad (57)$$

where $l, h \in \mathbb{R}$ are learnable thresholds and $\mathbf{x} \in \mathbb{R}^2$ is a two-dimensional input with components $\mathbf{x}_1, \mathbf{x}_2$. This model corresponds to a fixed-structure decision tree whose internal splits are defined by l and h . Classical approaches determine such thresholds by exhaustive or randomized search over candidate splits (cf. Breiman et al., 1984). SoftJAX, by contrast, renders the classifier differentiable, enabling l and h to be optimized directly via gradient descent. Figure 15(b, c) shows the thresholds quickly converging to a feasible solution within 10 steps, with accuracy reaching 100%.

We also demonstrate `@sj.st`: since decision trees typically produce hard class assignments, we decorate the loss with straight-through estimation, keeping the forward pass hard while routing gradients through the soft relaxation.

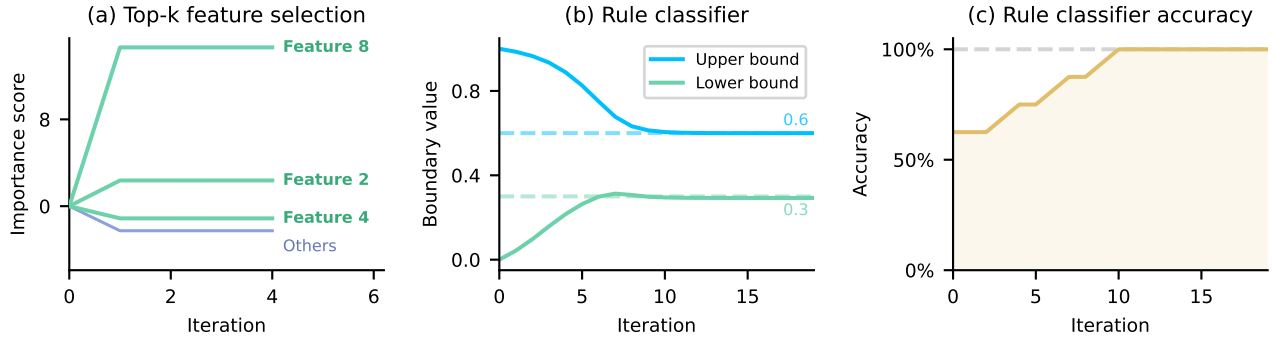


Figure 15. Optimization trajectories. **(a)** Learned importance scores for $n=10$ features; the three truly informative ones (Features 2, 4, 8) emerge after a single step. **(b, c)** Learned decision thresholds converging to a feasible solution and corresponding classification accuracy.

```
x = jnp.array([[0.2, 0.8], [0.5, 0.3], [0.9, 0.1], [0.4, 0.7], [0.1, 0.4], [0.2, 0.7], [0.4, 0.1], [0.4, 0.7],
               [0.7, 0.29], [0.3, 0.3], [0.61, 0.25], [0.4, 0.6], [0.0, 0.1], [0.5, 0.3], [0.4, 0.9], [0.1,
               0.57]])
labels = jnp.array([0.0, 1.0, 0.0, 1.0, 1.0, 0.0, 1.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 1.0, 1.0])

@sj.st
def rule_loss(params, x, labels, mode="smooth"):
    l, h = params[0], params[1]
    above = sj.greater(x, l, mode=mode)
    below = sj.less(x, h, mode=mode)
    in_range = sj.logical_and(above, below)
    preds = sj.any(in_range, axis=-1)
    return ((preds - labels) ** 2).sum()

params = jnp.array([0.0, 1.0]) # start with wide range [0, 1]
print("Hard grad:", jax.grad(rule_loss)(params, x, labels, mode="hard"))
print("Soft grad:", jax.grad(rule_loss)(params, x, labels, mode="smooth"))

for _ in range(20):
    params = params - 0.01 * jax.grad(rule_loss)(params, x, labels)
print("Learned [l, h]:", params)
```

Output:

```
Hard grad: [0. 0.]
Soft grad: [-4.2777  1.4152]
Learned [l, h]: [0.2925 0.5999]
```

C. Input Standardization and Squashing

All axiswise operators in SoftJAX and SoftTorch optionally preprocess the input $\mathbf{x} \in \mathbb{R}^n$ via a *standardize-and-squash* transform, following [Cuturi et al. \(2019\)](#). This serves two purposes: (i) it maps $\mathbf{x}$ into $[0, 1]$, matching the range of the grid-like target marginal $\mathbf{y} = \frac{1}{n-1}[0, \dots, n-1]^\top$ used in optimal transport, improving numerical stability; and (ii) it makes the softness parameter τ independent of the scale of $\mathbf{x}$.

Forward transform. Given the input $\mathbf{x}$, we first standardize and then squash with the logistic sigmoid σ , i. e.,

$$\mu = \text{mean}(\mathbf{x}), \quad s = \text{std}(\mathbf{x}), \quad \tilde{\mathbf{x}} = \sigma\left(\frac{\mathbf{x} - \mu}{s}\right), \quad (58)$$

where μ and s are computed along the specified axis. The transformed input $\tilde{\mathbf{x}} \in (0, 1)^n$ is then passed to the axiswise operator.

Inverse transform. For value-returning operators (e. g., `sort`), the output $\tilde{\mathbf{y}} \in (0, 1)^n$ is then mapped back to the original scale. For this we apply the inverse, by first unsquashing via the logit function, and then destandardizing, i. e.,

$$\mathbf{y} = \text{logit}(\tilde{\mathbf{y}}) \cdot s + \mu, \quad \text{where} \quad \text{logit}(p) = \log \frac{p}{1-p}. \quad (59)$$

The exception is SmoothSort (Section 4.3), which skips standardize-and-squash. This is because it is the only method which can return values larger than the largest input element (or smaller than the smallest input element) as the output is not a convex combination of inputs. Hence with standardize-and-squash, the output would not always be in the interval $(0, 1)$, which can lead to undefined outputs of the unsquashing operation.

D. Extended Axiswise Operators

D.1. Solving optimization problems

Optimal transport. In SoftJAX, for `smooth`-mode OT, we use Sinkhorn iterations implemented via `ott-jax` ([Cuturi et al., 2022](#)) by default. Alternatively, L-BFGS as suggested in ([Blondel et al., 2018](#)), implemented via the `Optimistix` library ([Rader et al., 2024](#)), can be used. For `c0`, `c1`, and `c2` modes, we solve the dual OT problems (see Section D.2 for their formulations) using L-BFGS via `Optimistix`. Both libraries support implicit differentiation, which reduces memory usage and enables adaptive stopping criteria for optimization loops. In SoftTorch, we solve the optimal transport problems using POT ([Flamary et al., 2021; 2024](#)).

Projection onto the unit simplex. In `smooth` mode, this reduces to a simple call to a softmax function. In `c0` mode, we use the well-known $O(n \log n)$ algorithm of ([Wang & Carreira-Perpiñán, 2013](#)). For `c1` and `c2` modes, we derive closed-form solutions to the Bregman projection onto the unit simplex ([Bregman, 1967](#)), namely a quadratic formula for $p = 3/2$ (`c1`) and Cardano’s method for $p = 4/3$ (`c2`), both running in $O(n \log n)$.

Projection onto the permutahedron. We follow ([Blondel et al., 2020](#)), who reduce the Euclidean and log-KL projections onto the permutahedron to isotonic optimization, which is in turn solved in $O(n \log n)$ via the pool adjacent violators (PAV) algorithm.

For SoftJAX, we reimplement the PAV algorithm in pure JAX, as the original NumPy code is incompatible with `jax.vmap/jax.jit`. In SoftTorch, we rely Numba-JIT for acceleration.

Moreover, we extend FastSoftSort to p -norm regularization following [Sander et al. \(2023\)](#). Note, that [Sander et al. \(2023\)](#) solve the PAV subproblem via bisection, we replace this with closed-form solutions to a quadratic formula for `c1` and Cardano’s method for `c2`.

For SmoothSort (Section 4.3) we solve the dual of the entropically-regularized LP via L-BFGS ([Rader et al., 2024](#)). A custom VJP on the solver computes gradients via an adjoint saddle-point system. Note that SmoothSort is currently SoftJAX only.

D.2. Dual OT problems

For marginals $\mathbf{a} \in \Delta_n$, $\mathbf{b} \in \Delta_m$, cost matrix $C \in \mathbb{R}^{n \times m}$, and dual potentials $\mathbf{f} \in \mathbb{R}^n$, $\mathbf{g} \in \mathbb{R}^m$, the dual of the entropy-regularized OT problem can be written as

$$\max_{\mathbf{f} \in \mathbb{R}^n, \mathbf{g} \in \mathbb{R}^m} \langle \mathbf{f}, \mathbf{a} \rangle + \langle \mathbf{g}, \mathbf{b} \rangle - \tau \sum_{ij} \exp\left(\frac{f_i + g_j - C_{ij}}{\tau}\right). \quad (60)$$

and the corresponding optimal primal solution can be recovered via

$$(\Gamma_\tau^*)_{ij} = \exp\left(\frac{f_i^* + g_j^* - C_{ij}}{\tau}\right). \quad (61)$$

The dual of the Euclidean regularized OT problem can be written as

$$\max_{\mathbf{f} \in \mathbb{R}^n, \mathbf{g} \in \mathbb{R}^m} \langle \mathbf{f}, \mathbf{a} \rangle + \langle \mathbf{g}, \mathbf{b} \rangle - \frac{1}{2\tau} \sum_{ij} (f_i + g_j - C_{ij})_+^2 \quad (62)$$

and the corresponding optimal primal solution can be recovered via

$$(\Gamma_\tau^*)_{ij} = \frac{1}{\tau} (f_i^* + g_j^* - C_{ij})_+. \quad (63)$$

The dual of the p-norm regularized OT problem can be written as

$$\max_{\mathbf{f} \in \mathbb{R}^n, \mathbf{g} \in \mathbb{R}^m} \langle \mathbf{f}, \mathbf{a} \rangle + \langle \mathbf{g}, \mathbf{b} \rangle - \frac{\tau^{1-q}}{q} \sum_{ij} (f_i + g_j - C_{ij})_+^q, \quad (64)$$

where $q = \frac{p}{p-1}$. The corresponding optimal primal solution can be recovered via

$$(\Gamma_\tau^*)_{ij} = \left(\frac{f_i^* + g_j^* - C_{ij}}{\tau}\right)_+^{\frac{1}{p-1}}. \quad (65)$$

D.3. Elementwise operators as special case

While elementwise and axiswise soft operators are treated as distinct categories in the main text, they are closely related. Specifically, the elementwise soft surrogates described in Section 3 can be recovered as special cases of the axiswise operators when $n = 2$. Here, we demonstrate that applying axiswise operators to a two-element vector $\mathbf{x} = [0, x]^\top$ yields the soft Heaviside functions derived earlier. To distinguish the mode-specific sigmoidals in this section, we introduce superscripts for the smoothness class. The smooth mode is $H_\tau^\infty(x) := \sigma(x/\tau) = e^{x/\tau}/(1 + e^{x/\tau})$. All piecewise modes share the structure

$$H_\tau^k(x) := \begin{cases} 0 & \text{if } x < -\tau, \\ g^k(x/\tau) & \text{if } |x| \leq \tau, \\ 1 & \text{if } x > \tau, \end{cases} \quad (66)$$

with interpolation functions $g^k : [-1, 1] \rightarrow [0, 1]$

$$g^0(s) = \frac{1}{2} + \frac{s}{2}, \quad (\text{c0, piecewise linear, } \mathcal{C}^0) \quad (67)$$

$$g^1(s) = \frac{1}{2} + \frac{3s}{4} - \frac{s^3}{4}, \quad (\text{c1, Hermite cubic, } \mathcal{C}^1) \quad (68)$$

$$g^2(s) = \frac{1}{2} + \frac{15s}{16} - \frac{5s^3}{8} + \frac{3s^5}{16}. \quad (\text{c2, Hermite quintic, } \mathcal{C}^2) \quad (69)$$

The p -norm simplex projections at $n = 2$ yield distinct $\mathcal{C}^1$ and $\mathcal{C}^2$ sigmoidals, which we denote $\tilde{H}_\tau^1, \tilde{H}_\tau^2$ (same piecewise structure with $\tilde{g}^k$ replacing g^k)

$$\tilde{g}^1(s) = \frac{(s + \sqrt{2 - s^2})^2}{4}, \quad (p=3/2 \text{ p-norm, } \mathcal{C}^1) \quad (70)$$

$$\tilde{g}^2(s) = (t + s/2)^3, \quad t = -|s| \sinh\left(\frac{1}{3} \operatorname{arcsinh}\left(\frac{-2}{s^2|s|}\right)\right). \quad (p=4/3 \text{ p-norm, } \mathcal{C}^2) \quad (71)$$

These smoothness classes are confirmed for general n in Theorem 1.

Reduction of soft argmax. Consider the soft argmax operator applied to $\mathbf{x} = [0, x]^\top$. The output is a probability distribution $\mathbf{p} \in \Delta_2$ over the indices. Since $p_0 + p_1 = 1$, the distribution is fully characterized by p_1 , which is a sigmoidal function of x/τ for each regularizer, i. e.,

$$\arg \max_\tau([0, x]^\top) = [1 - p_1, p_1]^\top. \quad (72)$$

For entropic regularization, $\arg \max$ reduces to the standard softmax, and hence we recover the sigmoid

$$p_1 = \sigma(x/\tau) = H_\tau^\infty(x). \quad (73)$$

For p -norm regularization, the projection of $[0, x]^\top$ onto the unit simplex solves $\arg \min_{\mathbf{p} \in \Delta_2} -\langle [0, x]^\top, \mathbf{p} \rangle + \frac{\tau}{p} \sum_i p_i^p$.

With $p = 2$ (c0 mode), the first-order condition gives

$$p_1 = \frac{1}{2} + \frac{x}{2\tau} = H_\tau^0(x), \quad (74)$$

directly recovering the elementwise c0 sigmoidal.

With $p = 3/2$ (c1 mode), we get the first-order condition $\sqrt{p_1} - \sqrt{1-p_1} = x/\tau$, which can be solved via the quadratic formula by $p_1 = \tilde{H}_\tau^1(x)$. This is another sigmoidal function in $\mathcal{C}^1$ with $p'_1 = 0$ at $x = \pm\tau$.

With $p = 4/3$ (c2 mode), we get the first-order condition $p_1^{1/3} - (1-p_1)^{1/3} = x/\tau$, which reduces to a depressed cubic. That can be solved by Cardano's formula, giving $p_1 = \tilde{H}_\tau^2(x)$. This is yet another sigmoidal function in $\mathcal{C}^2$. Note that H^1 , H^2 are not special cases of an axiswise operator, but provide the same smoothness guarantees with simpler expressions. Theorem 1 confirms that the $\mathcal{C}^1$ and $\mathcal{C}^2$ smoothness classes of $\tilde{H}_\tau^1$ and $\tilde{H}_\tau^2$ extend to the general n -dimensional simplex and transport polytope projections.

Reduction of soft sort and OT. Similarly, we can analyze the behavior of soft sorting and optimal transport for $n = 2$. Let the input be $\mathbf{x} = [0, x]^\top$ with sorted anchors $\mathbf{y} = [0, 1]^\top$ and balanced marginals $\mathbf{a} = \mathbf{b} = [\frac{1}{2}, \frac{1}{2}]^\top$. The cost matrix is denoted as $C_{ij} = (x_i - y_j)^2$. The OT solution then takes the form

$$P_\tau^* = \begin{pmatrix} p_1 & 1 - p_1 \\ 1 - p_1 & p_1 \end{pmatrix}, \quad \Gamma = P/2, \quad (75)$$

which only leaves one degree of freedom $p_1 \in [0, 1]$. The cost simplifies to $\langle C, \Gamma \rangle = \frac{1+x^2}{2} - xp_1$, hence the regularized OT problem reduces to a scalar optimization in p_1 .

For entropy-regularized OT, the first-order condition $\log(p_1/(1-p_1)) = x/\tau$ gives

$$p_1 = \sigma(x/\tau) = H_\tau^\infty(x), \quad (76)$$

recovering the logistic sigmoid, identical to the simplex projection in the previous paragraph. For Euclidean regularization ($p=2$), the first-order condition yields

$$p_1 = \frac{1}{2} + x/\tau = H_{\tau/2}^0(x), \quad (77)$$

matching the simplex result $H_\tau^0(x)$ up to a factor of 2 in softness. For $p=3/2$, the first-order condition gives

$$\sqrt{p_1} - \sqrt{1-p_1} = \sqrt{2} x/\tau, \quad (78)$$

and for $p=4/3$,

$$p_1^{1/3} - (1-p_1)^{1/3} = 2^{1/3} x/\tau, \quad (79)$$

recovering the results of the previous paragraph up to softness scaling as $\tilde{H}_{\tau/\sqrt{2}}^1(x)$ and $\tilde{H}_{\tau/\sqrt[3]{2}}^2(x)$, respectively.

Softness convention. In our sigmoidal definitions, the piecewise polynomial modes (Equation (6)) transition from zero to one on the interval $[-\tau, \tau]$. In contrast, the smooth mode reaches near-saturation only at $\approx \pm 5\tau$. Therefore, we introduce a scaling factor for the piecewise polynomial modes as $g(x/(5\tau))$, which widens their transition to $[-5\tau, 5\tau]$, matching that of the smooth mode.

For axiswise operators, we do not add any mode-dependent scaling constants. As a result, the same τ value produces different effective softness across modes and problem sizes n for axiswise operators. For users needing comparable behavior across modes or sizes, we recommend measuring the normalized entropy $H(\mathbf{P})/\log n$ of the soft permutation matrix and calibrating τ accordingly.

D.4. Smoothness of p -norm regularized projections

The preceding section verifies the smoothness classes $\mathcal{C}^1$ and $\mathcal{C}^2$ of the p -norm sigmoidals $\tilde{H}_\tau^1$ and $\tilde{H}_\tau^2$ for $n = 2$. We now prove that these smoothness classes hold for the general n -dimensional simplex projection and for the p -norm regularized optimal transport plan.

Theorem 1 (Smoothness of p -norm regularized projections). *Let $n \geq 2$, $1 < p \leq 2$, $\tau > 0$, and let $q = p/(p-1)$ be the conjugate exponent with q a positive integer. Define $k := q - 2$.*

(i) Unit simplex. The p -norm regularized projection $\Pi_\tau : \mathbb{R}^n \rightarrow \Delta_n$,

$$\Pi_\tau(\mathbf{x}) := \arg \max_{\mathbf{p} \in \Delta_n} \langle \mathbf{x}, \mathbf{p} \rangle - \frac{\tau}{p} \|\mathbf{p}\|_p^p, \quad (80)$$

is a $\mathcal{C}^k$ map. For entropic regularization $\sum_j p_j \log p_j$, the projection Π_τ is $\mathcal{C}^\infty$.

(ii) Transport polytope. For $m \geq 2$, marginals $\mathbf{a} \in \Delta_n$, $\mathbf{b} \in \Delta_m$ and cost matrix $C \in \mathbb{R}^{n \times m}$, the p -norm regularized optimal transport plan

$$\Gamma_\tau^*(C) := \arg \min_{\Gamma \in U(\mathbf{a}, \mathbf{b})} \langle \Gamma, C \rangle + \frac{\tau}{p} \sum_{ij} |\Gamma_{ij}|^p \quad (81)$$

is a $\mathcal{C}^k$ map of C at every cost matrix for which the support $\{(i, j) : \Gamma_{ij}^* > 0\}$ forms a connected bipartite graph. For entropic regularization $\sum_{ij} \Gamma_{ij} (\log \Gamma_{ij} - 1)$, the transport plan Γ_τ^* is a $\mathcal{C}^\infty$ map of C unconditionally, since the support is always the full bipartite graph.

Moreover, the bound $\mathcal{C}^k$ is sharp: Π_τ is not $\mathcal{C}^{k+1}$, and Γ_τ^* is not $\mathcal{C}^{k+1}$. In particular, $c0$ ($p=2$) gives $\mathcal{C}^0$, $c1$ ($p=3/2$) gives $\mathcal{C}^1$, $c2$ ($p=4/3$) gives $\mathcal{C}^2$, and $smooth$ gives $\mathcal{C}^\infty$.

Since the cost matrix in the sorting application is $C_{ij} = (x_i - y_j)^2$, which is $\mathcal{C}^\infty$ in $\mathbf{x}$, the transport plan Γ_τ^* inherits the same $\mathcal{C}^k$ regularity as a function of the input $\mathbf{x}$ by the chain rule. The connectivity condition is generically satisfied, i. e., it holds for all cost matrices outside a set of measure zero.

Proof. The proof is based on the implicit function theorem applied to the optimality conditions of the optimization problems. Both parts rely on the positive-part power function, defined as $\varphi : \mathbb{R} \rightarrow [0, \infty)$ by $\varphi(t) := (t/\tau)_+^{k+1}$, of which all derivatives up to order k exist and are continuous, therefore $\varphi \in \mathcal{C}^k(\mathbb{R})$.

Part (i): Unit simplex. The KKT conditions for the simplex-constrained problem are

$$x_i - \tau(p_i^*)^{p-1} - \nu + \mu_i = 0, \quad 1 \leq i \leq n, \quad (82)$$

$$\sum_{i=1}^n p_i^* = 1, \quad (83)$$

$$p_i^* \geq 0, \quad \mu_i \geq 0, \quad \mu_i p_i^* = 0, \quad (84)$$

where ν is the multiplier for the equality constraint $\mathbf{1}^\top \mathbf{p} = 1$ and μ_i are the multipliers for the non-negativity constraints. When $p_i^* > 0$, complementary slackness gives $\mu_i = 0$, so the stationarity condition reduces to $p_i^* = ((x_i - \nu)/\tau)^{1/(p-1)} =$

$\varphi(x_i - \nu)$. When $p_i^* = 0$, the stationarity condition gives $\mu_i = \nu - x_i \geq 0$, i. e., $x_i \leq \nu$. In both cases $p_i^* = \varphi(x_i - \nu)$, and substituting into the equality constraint yields

$$\sum_{i=1}^n \varphi(x_i - \nu) = 1. \quad (85)$$

Define $h: \mathbb{R} \times \mathbb{R}^n \rightarrow \mathbb{R}$ by $h(\nu, \mathbf{x}) := \sum_{i=1}^n \varphi(x_i - \nu) - 1$. As a finite sum of translates of φ , the function h is $\mathcal{C}^k$ in $(\nu, \mathbf{x})$. The partial derivative with respect to ν is

$$\frac{\partial h}{\partial \nu} = - \sum_{i=1}^n \varphi'(x_i - \nu), \quad (86)$$

which is strictly negative at any $(\nu, \mathbf{x})$ satisfying $h = 0$. Indeed, $\sum_i \varphi(x_i - \nu) = 1 > 0$ implies that at least one index i has $x_i > \nu$, and for any such index $\varphi'(x_i - \nu) > 0$.

For $k \geq 1$ (i. e., $p < 2$), the Implicit Function Theorem for $\mathcal{C}^k$ maps yields that ν is a $\mathcal{C}^k$ function of $\mathbf{x}$. Furthermore, the implicit $\nu(\mathbf{x})$ is unique and globally defined, since $h(\cdot, \mathbf{x})$ is strictly decreasing (it satisfies $\partial h / \partial \nu < 0$ at every root). Each component $p_i^*(\mathbf{x}) = \varphi(x_i - \nu(\mathbf{x}))$ is then $\mathcal{C}^k$ as a composition of $\mathcal{C}^k$ functions, establishing $\Pi_\tau \in \mathcal{C}^k$. For entropic regularization, $\Pi_\tau(\mathbf{x}) = \exp(\mathbf{x}/\tau) / \sum_j \exp(x_j/\tau)$, which is $\mathcal{C}^\infty$.

Part (ii): Transport polytope. The argument follows the same structure as Part (i). From the dual formulation (Section D.2), the optimal transport plan satisfies $\Gamma_{ij}^* = \varphi(f_i^* + g_j^* - C_{ij})$, where $(\mathbf{f}^*, \mathbf{g}^*)$ are the dual potentials. The marginal constraints $\sum_j \varphi(f_i^* + g_j^* - C_{ij}) = a_i$ and $\sum_i \varphi(f_i^* + g_j^* - C_{ij}) = b_j$ define, after fixing $g_m = 0$ to break the shift symmetry, a $\mathcal{C}^k$ implicit system of $n + m - 1$ equations in $n + m - 1$ unknowns. The Jacobian is non-singular if and only if the bipartite support graph $\{(i, j) : \Gamma_{ij}^* > 0\}$ is connected, which is the standard non-degeneracy condition for optimal transport. Under this condition, the implicit function theorem gives $\Gamma_\tau^*(C) \in \mathcal{C}^k$. For entropic regularization, $\Gamma_{ij}^* = \exp((f_i^* + g_j^* - C_{ij})/\tau) > 0$ for all (i, j) , so the support is always connected and the same argument yields $\Gamma_\tau^* \in \mathcal{C}^\infty$.

Sharpness. For any $n \geq 2$, restricting Π_τ to inputs of the form $\mathbf{x} = (x_1, x_2, -M, \dots, -M)$ with M large enough that only the first two components are active reduces the projection to the $n = 2$ case. The preceding section shows that this two-dimensional projection is the sigmoidal $p_1 = \tilde{H}_\tau^k(x)$, whose $(k+1)$ -th derivative is discontinuous at $|x| = \tau$. Since this is a restriction of Π_τ , the full map cannot be $\mathcal{C}^{k+1}$. The same embedding applies to Γ_τ^* via the $n = m = 2$ reduction shown in the preceding section. $\square$

D.5. Gating- vs integration-like behavior of axiswise operators

In the main text, we have seen that there are two principled ways to get a soft relu surrogate from a soft sigmoidal, either by integration

$$\text{relu}_\tau(x) := \int_0^x H_\tau(t) dt, \quad (87)$$

which, in the case of the `smooth` regularizer, is simply a Softplus function, or via a gating mechanism

$$\text{relu}_\tau(x) := H_\tau(x) \cdot x, \quad (88)$$

which, in the case of an `smooth` regularizer, is simply a SiLU function,

We now observe that a similar mechanism is at play for axiswise operators: To get a soft surrogate for the sort operator, we “gate” the argsort operator with the input, i.e.

$$\text{sort}_\tau(\mathbf{x}) := \arg \text{sort}_\tau(\mathbf{x}) \mathbf{x} = P_\tau^*(\mathbf{x}) \mathbf{x}. \quad (89)$$

Intuitively, this means that the sort operator defined in this way behaves qualitatively similar to the SiLU function, which has a characteristic minimum, in contrast to the monotonically increasing Softplus function.

A natural question is whether a version of the sort operator exists that behaves similarly to the Softplus function. Unfortunately, this would require integrating the multivariate argsort function. However, we are often anyway mainly interested in the *gradient* of the soft sort surrogate, while leaving the forward pass hard via straight-through estimation. The gradient (or, more precisely, the Jacobian) is extremely simple to compute. It is the function we would integrate, i. e., the argsort operator matrix.

In code, similar to the straight-through trick, we can manipulate our sort implementation to return the Jacobian of the integration-style version by using a stop-gradient operation on the argsort operator, i. e.,

```
soft_idx = jax.lax.stop_gradient(sj.argsort(x))
sorted_values = sj.take_along_axis(x, soft_idx)
```

This option is available via a simple flag, and it can be easily combined with straight-through estimation to get the hard forward pass and integration-style backward pass, e. g.,

```
sorted_values_st = sj.sort_st(x, gated_grad=False)
```

D.6. Entropy-regularized dual of permutahedron projection

Primal LP. The optimization problem over the permutahedron as stated in the main text is given by

$$\max_{\mathbf{p} \in \mathcal{P}(\mathbf{z})} \langle \mathbf{y}, \mathbf{p} \rangle. \quad (90)$$

We can equivalently characterize the permutahedron constraint $\mathbf{p} \in \mathcal{P}(\mathbf{z})$ through a set of majorization inequalities

$$\sum_{i=1}^k p_i^\downarrow \leq \sum_{i=1}^k z_i^\downarrow \quad \forall k: 1 \leq k < n \quad (91)$$

and the sum equality

$$\sum_{i=1}^n p_i = \sum_{i=1}^n z_i, \quad (92)$$

where $\mathbf{p}^\downarrow$ and $\mathbf{z}^\downarrow$ are sorted in descending order.

Reduction to ordered cone. W.l.o.g., we can restrict the search space to ordered $\mathbf{p}$ by enforcing

$$p_i \geq p_{i+1} \quad \forall i: 1 \leq i < n \quad (93)$$

and then reorder the optimal solution back via the inverse permutation that sorts $\mathbf{z}$. Crucially, on the ordered cone, the majorization inequalities become linear in $\mathbf{p}$

$$\sum_{i=1}^k p_i \leq \sum_{i=1}^k z_i^\downarrow \quad \forall k: 1 \leq k < n. \quad (94)$$

We can therefore rewrite the permutahedron LP as

$$\begin{aligned} & \max_{\mathbf{p} \in \mathbb{R}^n} \langle \mathbf{y}^\downarrow, \mathbf{p} \rangle \\ & \text{subject to } A\mathbf{p} \leq \mathbf{b} \\ & \quad \mathbf{1}^\top \mathbf{p} = c \\ & \quad D\mathbf{p} \geq 0, \end{aligned} \quad (95)$$

where we used $A \in \mathbb{R}^{(n-1) \times n}$, $\mathbf{b} \in \mathbb{R}^{(n-1)}$, $c \in \mathbb{R}$, $D \in \mathbb{R}^{(n-1) \times n}$, defined as

$$(A\mathbf{p})_k := \sum_{i=1}^k p_i \quad \forall 1 \leq k < n \quad (96)$$

$$b_k := \sum_{i=1}^k z_i^\downarrow \quad \forall 1 \leq k < n \quad (97)$$

$$c := \sum_{i=1}^n z_i \quad (98)$$

$$(D\mathbf{p})_i := p_i - p_{i+1} \quad \forall 1 \leq i < n. \quad (99)$$

Smoothing. The LP above is non-smooth in two ways. First, computing the majorization bounds $b_k = \sum_{i=1}^k z_i^\downarrow$ requires sorting $\mathbf{z}$, which is not a smooth operation. Second, the LP itself is a linear program whose solution jumps between vertices. We address both by (i) replacing $\mathbf{b}$ with a smooth relaxation $\tilde{\mathbf{b}}$, and (ii) adding entropic regularization on the inequality slack variables.

For the bounds, we can rewrite b_k as an optimization problem over subsets of indices, i. e.,

$$b_k = \sum_{i=1}^k z_i^\downarrow = \max_{\substack{S \subseteq [n] \\ |S|=k}} \sum_{i \in S} z_i. \quad (100)$$

We can then replace the hard maximum with a log-sum-exp, which leads to a smooth relaxation, i. e.,

$$\tilde{b}_k := \tau \log \sum_{\substack{S \subseteq [n] \\ |S|=k}} \exp\left(\frac{1}{\tau} \sum_{i \in S} z_i\right) = \tau \log e_k(e^{\mathbf{z}/\tau}), \quad (101)$$

where $e_k(\mathbf{x}) = \sum_{|S|=k} \prod_{i \in S} x_i$ sums all products of k distinct elements from $\mathbf{x}$. In the limit $\tau \rightarrow 0^+$, we have $\tilde{b}_k \rightarrow b_k$. Since $e_n(\mathbf{x}) = \prod_i x_i$, we directly get $\tilde{b}_n = \sum_i z_i = c$ thereby preserving the equality constraint. Note also that the relaxed bounds satisfy $\tilde{b}_k \geq b_k$, so the feasible set is a slight superset of the true permutahedron.

We next explain how the smoothed bounds can be computed in practice. For this we use the recurrence

$$E_0^{(j)} = 1, \quad E_k^{(0)} = 0 \quad \text{for } k \geq 1, \quad (102)$$

$$E_k^{(j)} = E_k^{(j-1)} + e^{z_j/\tau} \cdot E_{k-1}^{(j-1)}, \quad (103)$$

where $E_k^{(j)} = e_k(e^{z_1/\tau}, \dots, e^{z_j/\tau})$. This requires $O(n^2)$ time runtime. Note that we work in log-space (logaddexp) for numerical stability and use optimal online checkpointing (Stumm & Walther, 2010) to reduce memory from $O(n^2)$ to $O(n\sqrt{n})$.

In order to smoothen the LP, we introduce slack variables and add entropic regularization onto them using $\phi(t) = t \log(t) - t$, i. e.,

$$\begin{aligned} & \max_{\mathbf{p} \in \mathbb{R}^n, \mathbf{s} \in \mathbb{R}^{n-1}, \mathbf{d} \in \mathbb{R}^{n-1}} \langle \mathbf{y}^\downarrow, \mathbf{p} \rangle - \tau \sum_{k=1}^{n-1} \phi(s_k) - \tau \sum_{i=1}^{n-1} \phi(d_i) \\ & \text{subject to } A\mathbf{p} + \mathbf{s} = \tilde{\mathbf{b}} \\ & \quad \mathbf{1}^\top \mathbf{p} = c \\ & \quad D\mathbf{p} = \mathbf{d}. \end{aligned} \quad (104)$$

Dual problem and solution recovery. The smoothed primal problem has the dual problem

$$\begin{aligned} & \min_{\alpha \in \mathbb{R}^{n-1}, \beta \in \mathbb{R}^{n-1}, \nu \in \mathbb{R}} \langle \alpha, \tilde{\mathbf{b}} \rangle + \nu c + \tau \sum_{k=1}^{n-1} e^{-\alpha_k/\tau} + \tau \sum_{i=1}^{n-1} e^{-\beta_i/\tau} \\ & \text{subject to } \mathbf{y}^\downarrow - A^\top \alpha + D^\top \beta - \nu \mathbf{1} = 0. \end{aligned} \quad (105)$$

We now proceed by first eliminating ν via the equality constraint

$$\mathbf{0} = (\mathbf{y}^\downarrow + D^\top \beta - \nu \mathbf{1})_n \Rightarrow \nu(\beta) = y_n^\downarrow + (D^\top \beta)_n. \quad (106)$$

Now we define

$$\mathbf{u}(\beta) = \mathbf{y}^\downarrow + D^\top \beta - \nu(\beta) \mathbf{1}, \quad (107)$$

which satisfies $u_n(\beta) = 0$. Note that have $\mathbf{u}(\beta) = A^\top \alpha$, which means

$$\alpha_k(\beta) = u_k(\beta) - u_{k+1}(\beta) \quad \forall k: 1 \leq k < n. \quad (108)$$

Finally, substituting $(\alpha_k(\beta), \nu(\beta))$ into the dual gives an unconstrained optimization problem over β , which we can solve via L-BFGS.

In the last step we recover the primal solution from the dual solution $(\alpha^*, \beta^*, \nu^*)$. This is done via

$$\mathbf{s}^* = \exp(-\alpha^*/\tau) \quad (109)$$

$$\mathbf{d}^* = \exp(-\beta^*/\tau) \quad (110)$$

and $\mathbf{p}^*$ is recovered by first defining prefix sums

$$r_k := \tilde{b}_k - s_k^* = \sum_{i=1}^k p_i^* \quad \forall k: 1 \leq k < n, \quad (111)$$

and then computing $\mathbf{p}^*$ by differencing

$$p_1^* = r_1 \quad (112)$$

$$p_k^* = r_k - r_{k-1} \quad \forall k: 2 \leq k < n \quad (113)$$

$$p_n^* = c - r_{n-1}. \quad (114)$$

Backward pass. The solver uses a custom VJP that solves an adjoint system via conjugate gradients, with $O(n)$ cost per iteration due to the banded structure of the LP constraints. Gradients through the smooth bounds use standard reverse-mode autodiff through Equation (101).

E. Additional Benchmark Experiments

We include additional benchmarking experiments for elementwise operators (Figure 16) and axiswise operators (Figures 17 to 23). Each figure shows runtime, JIT compilation time, and peak memory consumption as a function of input array size, broken down by regularization mode (smooth, c0, c1, c2).

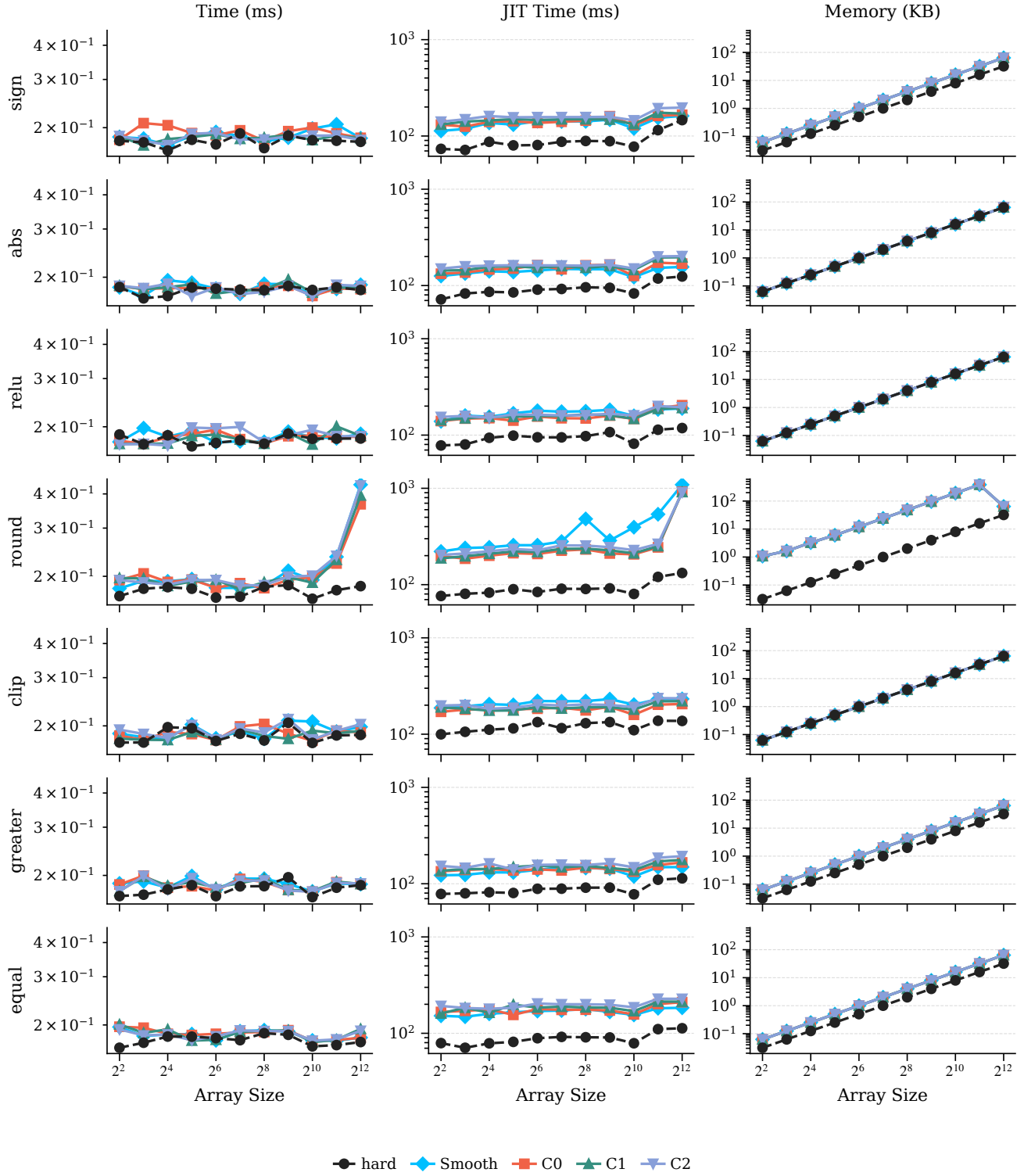
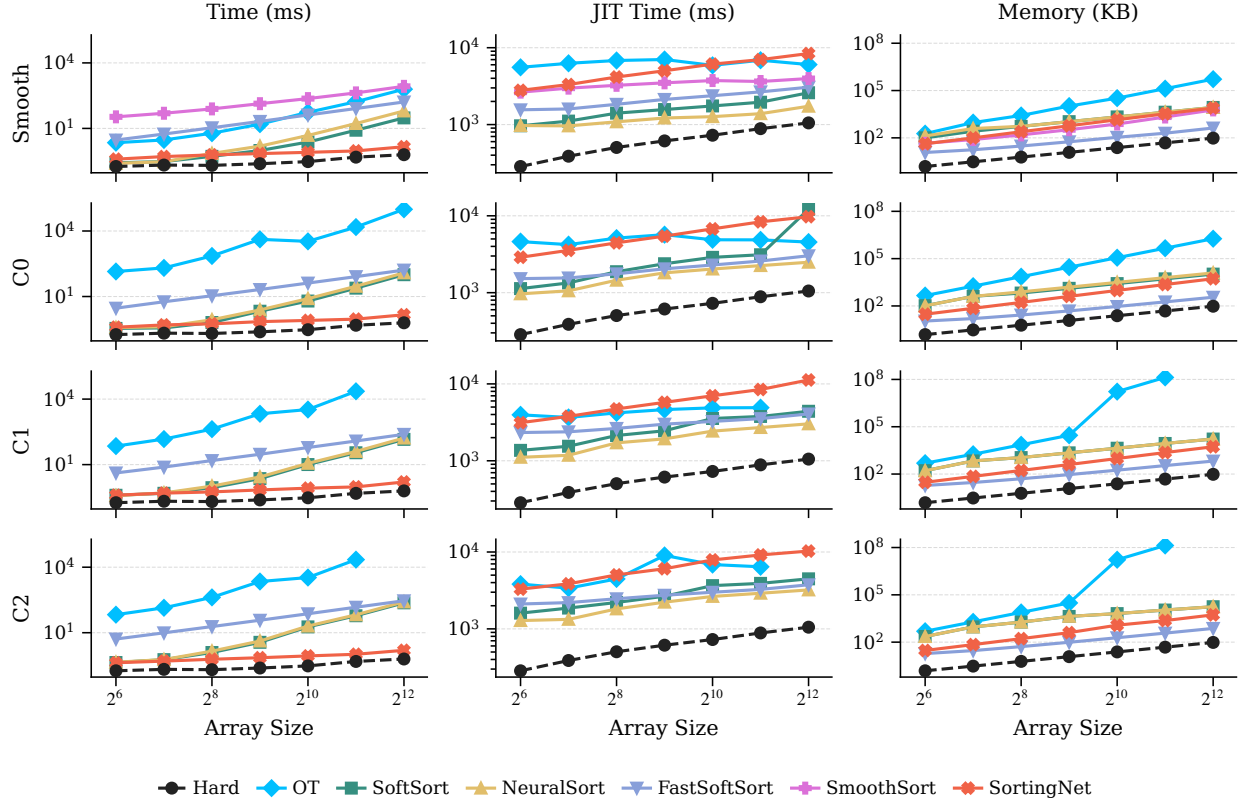
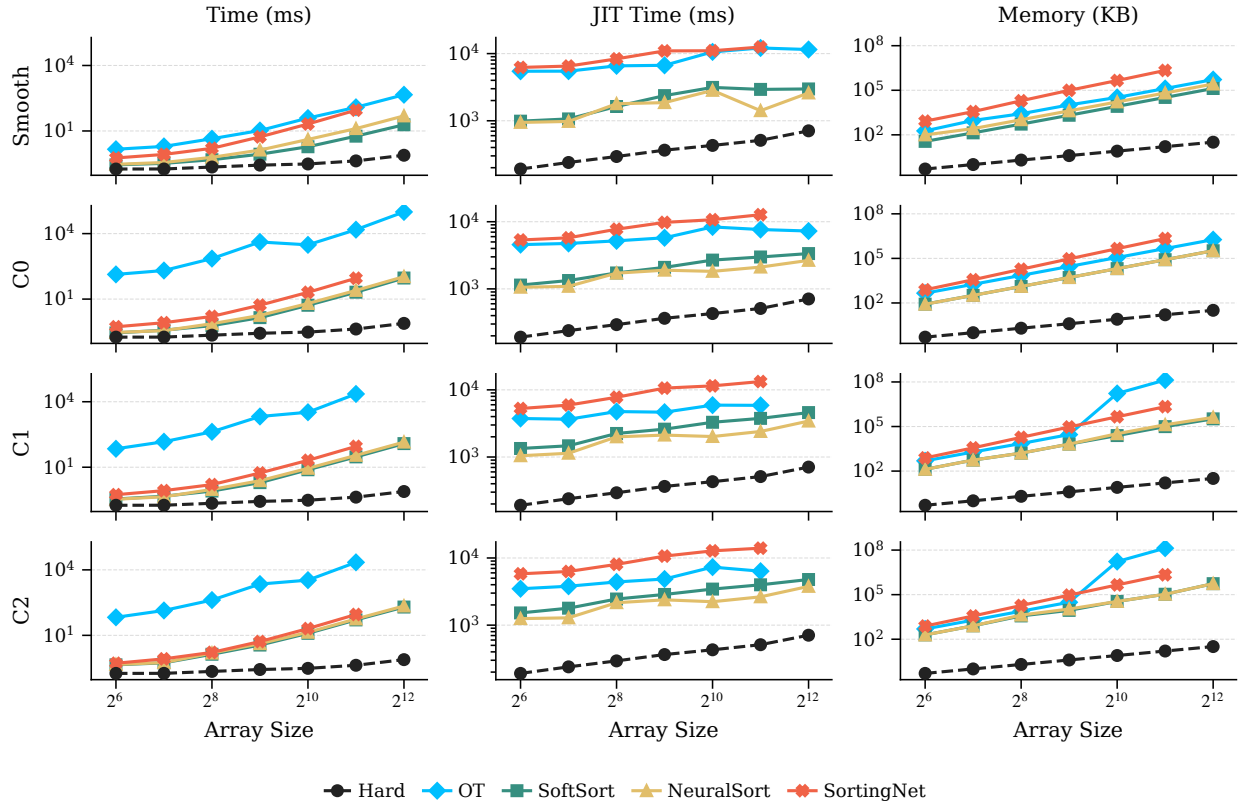
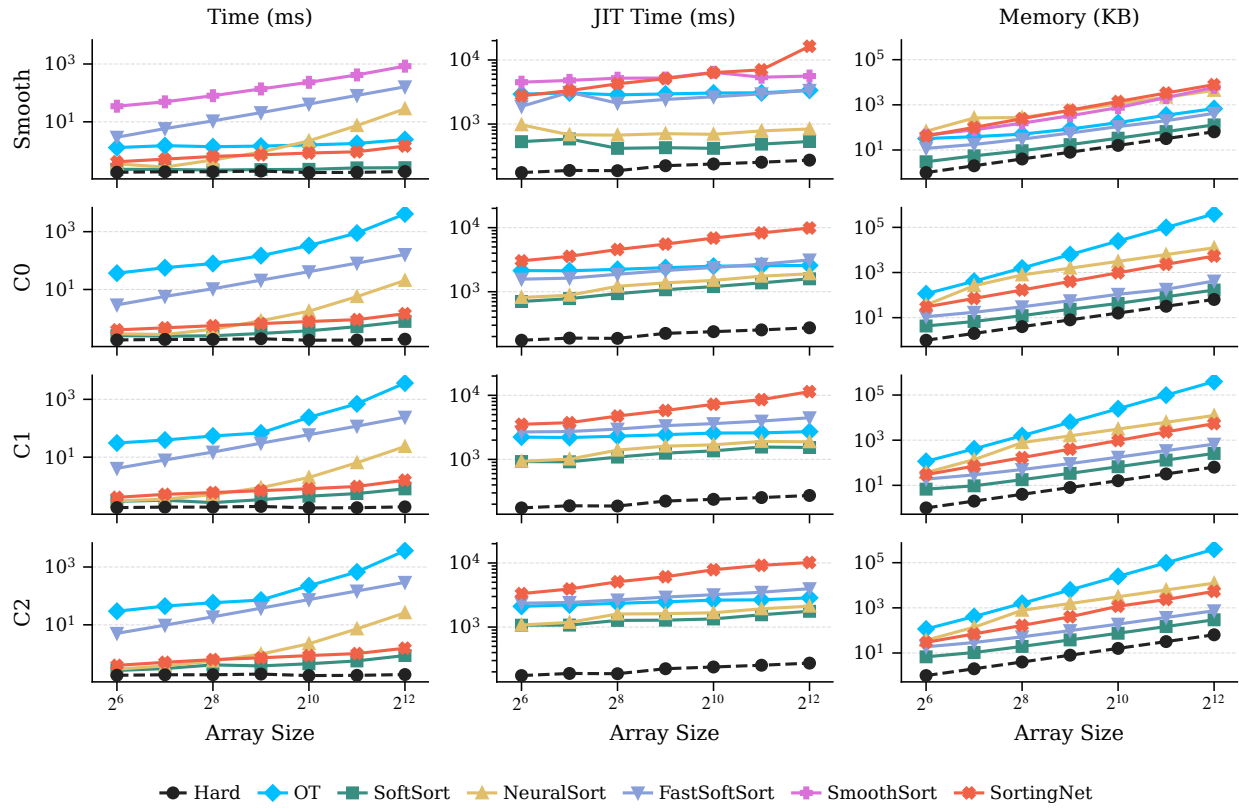
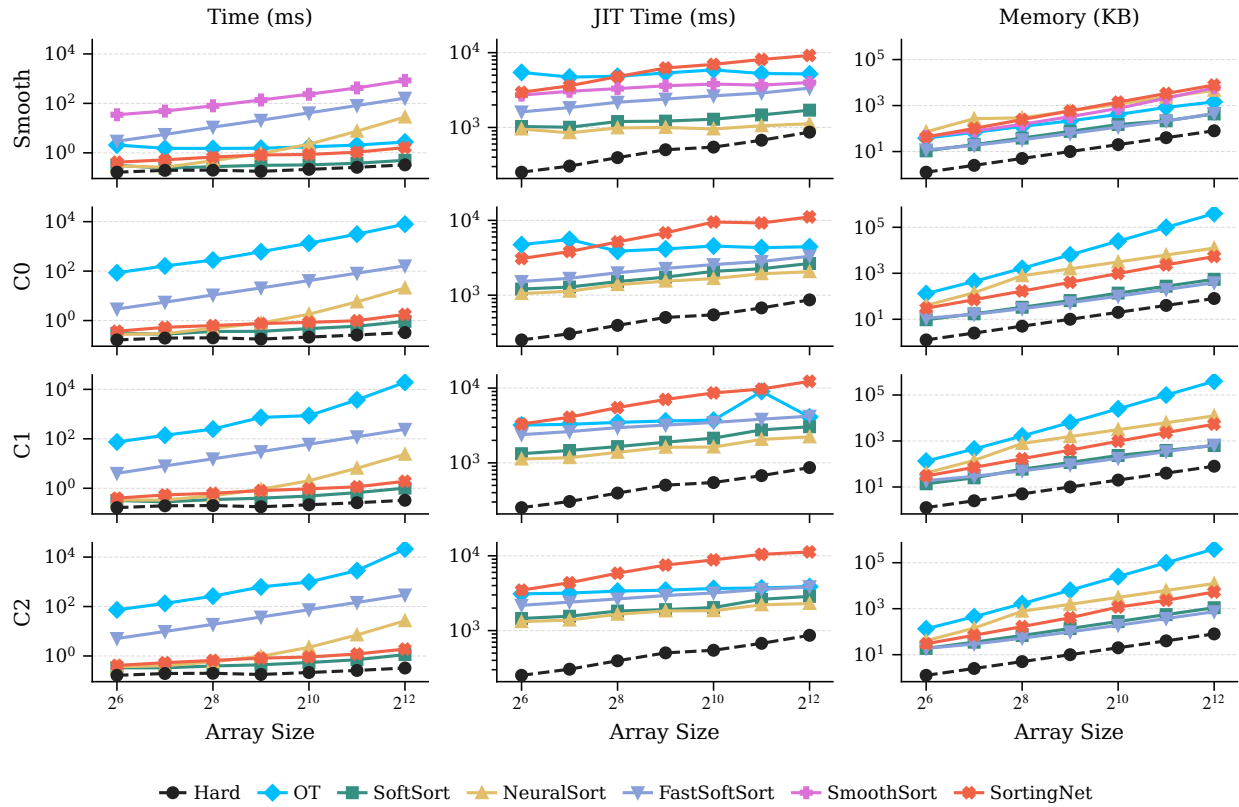
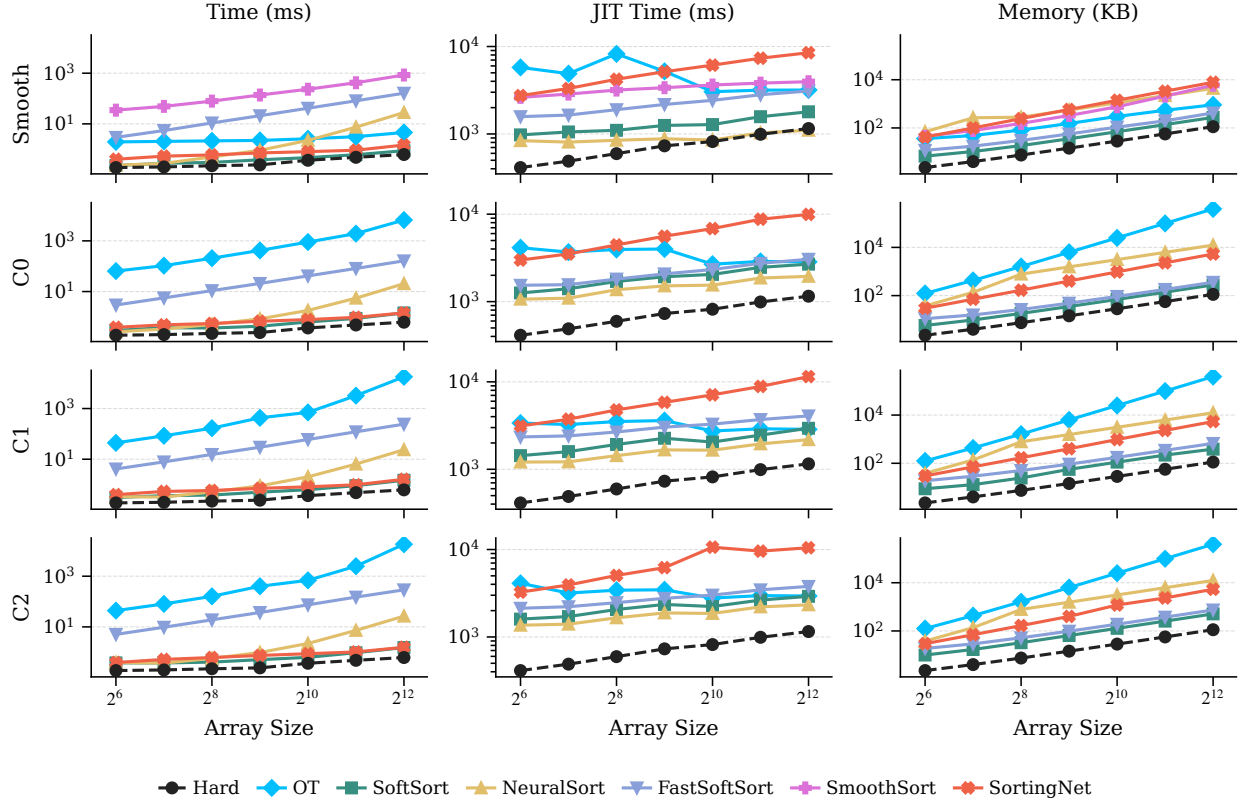
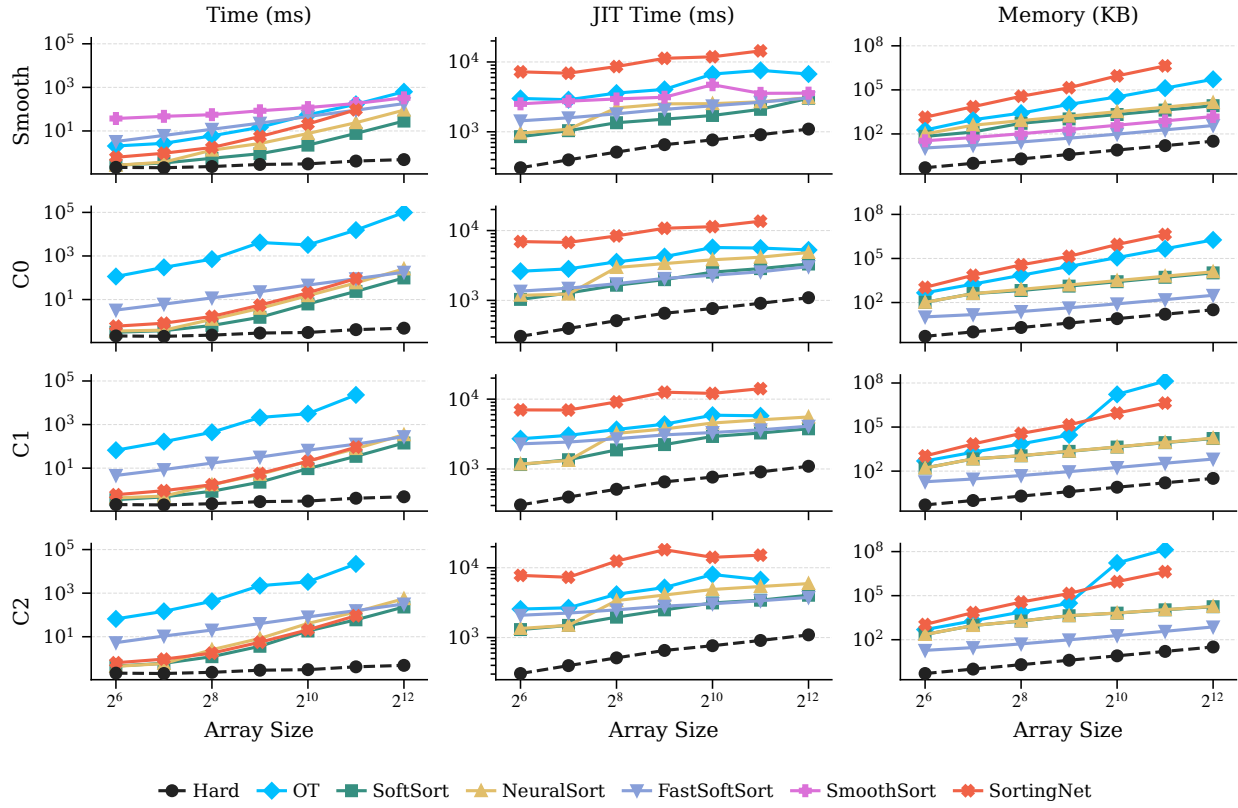
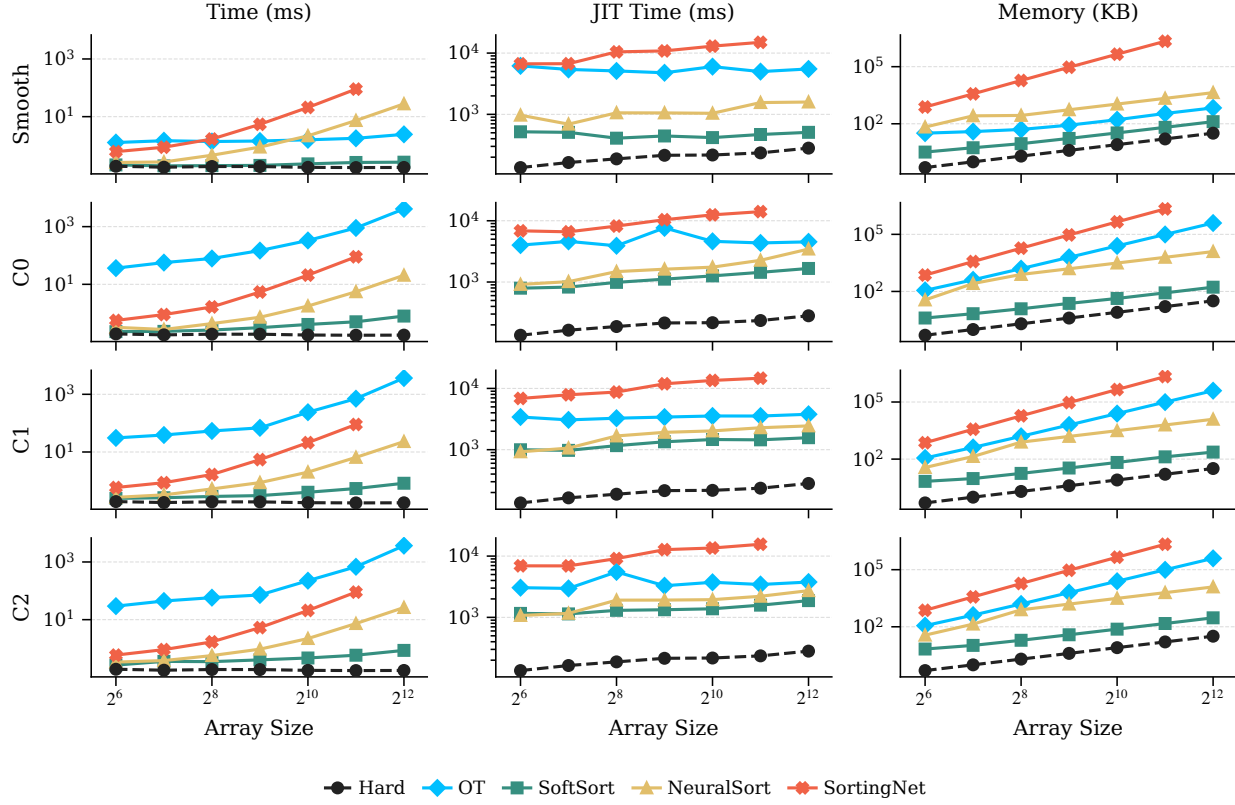


Figure 16. Benchmark results for elementwise operators.


 Figure 17. Benchmark results for `sj.sort`.

 Figure 18. Benchmark results for `sj.argsort`.


 Figure 19. Benchmark results for `sj.max`.

 Figure 20. Benchmark results for `sj.top_k`.


 Figure 21. Benchmark results for `sj.median`.

 Figure 22. Benchmark results for `sj.rank`.


 Figure 23. Benchmark results for `sj.argmax`.

F. Choosing a Method and Mode

The preceding benchmark results reveal clear tradeoffs between the available methods and modes. This section provides practical guidance for choosing among them.

Method selection. For most use cases, we recommend **NeuralSort** or **sorting networks** as the default methods. Sorting networks are the fastest method and are highly parallelizable on GPUs (see Figure 8). NeuralSort is also fast with low memory consumption and fast JIT compilation in SoftJAX, and produces fully smooth surrogates in `smooth` mode (by using soft abs instead of the hard sort in SoftSort’s cost matrix). Both methods support all operators, including soft index-producing functions like `argsort` and `argmax`.

When memory usage is the primary concern, **FastSoftSort** is the most memory-efficient method, as it avoids materializing the $n \times n$ permutation matrix entirely. However, FastSoftSort only produces sorted values and ranks, not soft indices (i. e., no `argsort` or `argmax`), and is not fully smooth due to its reliance on the hard sort for initialization.

For a more detailed comparison of all methods, we refer to the benchmark results in Figures 8 and 17 to 23.

Mode selection. For gradient-based optimization, we recommend `smooth` (C^∞) as the default mode. It produces the best-conditioned gradients and is compatible with higher-order differentiation (e. g., Hessian computation).

When a **bounded region of change** is desired, the piecewise modes `c1` and `c2` are good alternatives. These provide C^1 and C^2 smoothness respectively, with sparser gradients that are exactly zero outside the transition region. If Hessians are needed, `c2` should be preferred over `c1`.

The `c0` mode is generally **not recommended** for gradient-based optimization. It is only C^0 (continuous but not differentiable everywhere), and the axiswise operators that rely on optimization-based projections (e. g., OT) can be numerically unstable in `c0` mode, because the L2 regularization induces sparse transport plans with discontinuous optimality conditions (Theorem 1).

Softness parameter. For elementwise operators, all modes share an effective transition width of approximately 10τ (see Section D.3). For axiswise operators, the same τ value produces different effective softness across modes and problem sizes n . Users needing comparable behavior across modes or sizes can measure the normalized entropy $H(\mathbf{P})/\log n$ of the soft permutation matrix and calibrate τ accordingly. As a rule of thumb, smaller τ produces sharper approximations closer to the hard operator, at the cost of less informative gradients.